

Multicriteria decision-making-based framework for implementing DevOps practices: A fuzzy best-worst approach

Ankur Kumar¹  | Mohammad Nadeem¹  | Mohammad Shameem² 

¹Department of Computer Science, Aligarh Muslim University, Aligarh, India

²Department of Computer Science and Applications, Dr. Vishwanath Karad, MIT World Peace University, Pune, Maharashtra, India

Correspondence

Mohammad Nadeem, Department of Computer Science, Aligarh Muslim University, Aligarh 202002, India.

Email: nadeem.amu@gmail.com

Abstract

Increasingly, software organizations are implementing DevOps culture to benefit from it in terms of continuous testing, delivery, improvement, and so forth. Implementing DevOps is difficult due to a lack of understanding about the practices and their effective application for its effective implementation. This paper aims to explore different DevOps practices that can be implemented in software organizations. The study involves conducting a systematic literature review (SLR) to identify DevOps implementation practices, followed by the utilization of the fuzzy best-worst method (FBWM) to establish a taxonomy or classification of software practices. We have used an SLR to investigate the practices, and subsequently, the survey method was followed to validate the identified practices. Moreover, the best-worst method (BWM) was considered to evaluate the significance and develop the taxonomy of the practices. The results of this study extracted 19 practices that have been identified in the SLR process. The identified factors are further classified into six core DevOps lifecycle phases. The results of the BWM approach are shown. The outcomes of the study conclude that the proposed taxonomy of the practices could help DevOps practitioners and researchers effectively implement them in software development organizations.

KEYWORDS

DevOps, fuzzy best-worst method, MCDM, SLR

1 | INTRODUCTION

In today's digital world, organizations face unprecedented pressure to release quality software products with low development to rapidly meet the customer's demands.¹ The implementation of DevOps practices has emerged as a cultural shift that prioritizes delivering customer requirements by emphasizing enhanced collaboration, accelerated release cycles, and heightened automation.² DevOps refers to the joint effort of development and operations teams to achieve continuous software deployment and delivery.³ In the traditional software process, the Dev and Ops teams have to work in isolation and each team has their predefined specific tasks and needs to follow the hierarchy. In the traditional approach, the collaboration between the team members and customers is limited, that is, during requirements gathering and product release, which increases the challenges related to the requirements change management and customer feedback.⁴ Consequently, the traditional approaches often result in downtime and deployment delays due to the division between development and operations teams.⁵ By planning and synchronizing IT and development tasks throughout the CI/CD pipeline,⁶ DevOps minimizes these disruptions. DevOps oversees all aspects of the development lifecycle, starting from initial planning and extending to coding, testing, deploying services, ongoing monitoring, and maintaining customer engagement.⁷ This is accomplished by leveraging a continuous pipeline that automates manual and resource-

intensive practices while tracking previously performed tasks.⁸ The software organization perceives DevOps as a movement that seeks to improve communication, coordination, and integration between Developers and Operators.⁹ This movement encompasses both cultural and technological aspects. Thus, merely implementing technological changes, such as adopting tools and automation, is insufficient to generate significant value.¹⁰ Cultural changes are equally essential in achieving the desired outcomes.¹¹ Real-world examples of successful DevOps implementation can illustrate its transformative impact. For instance, Netflix embraced DevOps practices to enhance their software delivery, resulting in increased agility and faster time-to-market for new features.¹² Similarly, Amazon adopted DevOps methodologies to support its highly scalable infrastructure, enabling rapid and reliable software releases.¹³ These examples showcase the tangible benefits that organizations can achieve through the adoption of a DevOps culture.

DevOps is undergoing a transformative process that is reshaping the landscape of software development and operations. This evolution involves the integration of new disciplines and the extension of collaboration, automation, and feedback principles into previously unexplored domains. One notable outcome of this evolution is the emergence of Dev-SecOps, which integrates security practices into the DevOps workflow.¹⁴ By incorporating security from the beginning, organizations can proactively identify and address vulnerabilities throughout the software development lifecycle, reducing the risk of data breaches and enhancing system resilience. Another significant advancement is seen in the field of MLOps, which focuses on managing and deploying machine learning (ML) models in production environments. MLOps facilitates collaboration between data scientists and operations teams, enabling seamless integration of ML models into software systems. It also entails monitoring their performance and ensuring their reliability over time.¹⁵ In addition to DevSecOps and MLOps, there is a noteworthy convergence known as BizDevOps. This approach combines business, development, and operations perspectives to align organizational goals, customer requirements, and technical implementation. By promoting cross-functional collaboration and communication, BizDevOps streamlines processes, accelerates product delivery, and enhances customer satisfaction.¹⁶ These second-generation DevOps practices represent the evolution of the original movement, emphasizing the importance of holistic and collaborative approaches that incorporate security, ML, and business perspectives in modern software development and operations.

As DevOps is continuously evolving, software organizations differ in maturity and implementation strategies, making it challenging to deploy change efforts across organizations.¹⁷ The practitioners struggle with problems like collaboration in team contexts, integrating continuous delivery pipelines, leadership, and cultural change, and lacking the necessary skill set.¹⁸ DevOps can be simple for the organization, accepting the changes effectively and having the right frame of mind to make DevOps different.¹⁹ Many researchers have contributed to simplifying the implementation of DevOps by identifying the issues and workable solutions.²⁰ Senapathi et al²¹ investigated the benefits, challenges, leading drivers, and enablers of adopting DevOps. The benefits detailed in the study were promoting collaboration, higher quality code, more autonomy, natural communication, and knowledge sharing; boosting team morale; and increasing deployment frequency. A multivocal literature review was carried out by Amro et al²² to get opinions from the public on the relationship between capabilities and practices. The MLR discovered 37 practices and divided them into four groups based on their cultural, measurement, technological, and process capacities. McCarthy et al²³ conducted in-depth research and categorized DevOps implementations concerning developer interactions with the outsourced operations team, on-premises operations team, DevOps bridge team, and a DevOps team.

Implementing DevOps is not as simple as just buying a tool and ensuring everyone starts using it because DevOps is beyond a team of resources or persons that evolves daily.²⁴ DevOps cannot be implemented overnight due to the significant differences between DevOps and existing IT development practices.²⁵ To successfully establish a DevOps culture in software development organizations, specific practices must be employed to coordinate plans and objectives. Due to software development's intricate and time-consuming nature, such practices are crucial. Furthermore, each development practice contributes differently to achieving the DevOps objectives, and therefore, they should be assigned different weights based on their contributions. MCDM (multicriteria decision-making) techniques can assign different weights to development practices based on their contributions to achieving DevOps objectives. These techniques allow decision-makers to analyze and evaluate multiple criteria and make decisions that are based on multiple, often conflicting objectives.

However, it is true that the use of MCDM techniques for DevOps implementation is still in its early stages, and there is a need for more research in this area. Some studies have applied MCDM techniques to support decision-making in DevOps.²⁶ Akbar²⁷ utilized the fuzzy AHP to rank the 19 identified factors, whereas Noorani²⁸ employed the AHP approach to prioritize the 23 elements that affected DevOps. Anandya²⁹ also used AHP to prioritize the identified issues that influence DevOps. Although some studies have applied MCDM techniques to support decision-making in DevOps, studies examining DevOps from a software practices perspective are still lacking.³⁰ Here, 'practices for implementing DevOps' refer to the set of principles, methods, and techniques used to integrate software development (Dev) and information technology operations (Ops) within an organization. Understanding the role of software practices in DevOps is crucial for several reasons. Firstly, it enables organizations to identify and adopt best practices that promote collaboration, continuous integration, continuous delivery, and automation. By examining successful software practices, organizations can gain insights into the optimal utilization of tools like version control systems, build servers, automated testing frameworks, and deployment pipelines.³¹ Developing a taxonomy of software practices for implementing DevOps can be a complex and challenging task. Fuzzy best-worst method (FBWM) provides a structured framework for analysis that can help researchers systematically evaluate and prioritize different practices based on their performance in multiple criteria. In this study, after conducting a systematic literature review (SLR, Appendix 1) to identify practices for implementing DevOps, we utilized FBWM to create a taxonomy or classification of

software practices. This classification is based on the practices' relevance and effectiveness in achieving DevOps objectives.³² The results of applying the FBWM in this context can provide valuable insights into which software practices are most effective for implementing DevOps practices and can help organizations make informed decisions about which practices to adopt. Furthermore, the FBWM-generated taxonomy offers a valuable framework for organizing and comprehending the various software practices employed in DevOps implementation. Ultimately, this framework facilitates a more efficient and effective DevOps implementation by allowing organizations to prioritize practices with the highest impact on their objectives.

The goal of the current study will be accomplished with the execution of the steps described as follows: (1) to identify software practices heading toward a successful DevOps implementation,³³ (2) to validate the investigated software practices with the DevOps experts, and (3) to develop a prioritizing taxonomy using the FBWM for the software practices identified for effective implementation of DevOps. We developed the following research questions to achieve our goal:

RQ1: What core practices are needed to implement DevOps as discussed in the existing literature?

RQ1.1: How can the identified practices be classified?

RQ2: What is the significance of the identified DevOps practices in real-world practices?

RQ3: How can the priorities be assigned to the identified practices using the best-worst MCDM approach?

RQ4: How would the taxonomy of software practices be developed for DevOps adoption?

The remaining sections of the paper are organized as follows: Section 2 contains related work and motivation. Section 3 consists of the proposed research methodology. The findings of the research are discussed in Section 4. Section 5 contains a discussion of the proposed framework, and finally, Section 6 reports the conclusion and future work.

2 | RELATED WORK AND MOTIVATION

The research in DevOps applying the MCDM methods is reviewed in this section. DevOps is becoming more important in the literature of software development research due to its direct impact on the software development pace. Some of the significant studies that have been conducted in this domain are discussed below: In a study by Akbar,²⁷ 19 criteria were discovered by a thorough literature study. The discovered factors were subsequently validated among experts through a questionnaire survey investigation. After that, the determined success criteria were ranked using the fuzzy analytical hierarchy process (FAHP). The FAHP results show that the “people” category of the success criteria is the most important one. According to global rankings, the four top success criteria that must be taken into consideration for the effective deployment of DevOps methods are “emphasize culture rather than tools,” “evaluate your DevOps strategy,” “real-time feedback,” and “empowerment.”

Rafi et al.³⁴ expanded Hornbeek's work by utilizing the 15 DevOps testing techniques he covered in his paper. The pillars of DevOps adoption defined by CALMS (culture, automation, lean, measurement, and sharing) concepts were used to classify test practices. The results of a questionnaire-based study on DevOps test practices and their classification according to CALMS criteria were then gathered from software industry practitioners. To select the DevOps test practices, practitioners employed interpretive structure modeling (ISM) to uncover the relationships between the CALMS criteria.

Noorani²⁸ developed a readiness model by examining the variables that could have a favorable or unfavorable influence on DevOps operations in the software business. The detected factors are further categorized according to the firm's internal and external characteristics using the SWOT framework. Based on the Software Process Improvement Manifesto, 23 elements from the literature were identified and categorized into the four SWOT framework categories of strengths, weaknesses, opportunities, and threats. The established taxonomy of the factors reveals that “Strengths” (CW) is the highest ranked category of the identified factors. The findings show that for the development and operations teams to quickly create, test, and deploy each project, the DevOps environment must manage various components in each release and be committed in an automated manner.

The study of Anandya et al.²⁹ aims to identify the issues preventing DevOps deployment by examining the literature. Using the AHP technique, the problems identified were further validated and prioritized based on the opinions of IT professionals from Indonesian technology companies. As a result, the most pressing issues that need to be resolved for a successful DevOps implementation are a “highly bureaucratic deployment process,” “inadequate communication and collaboration issues,” and “lack of clear definition and strategic direction,” with values of 16.1%, 14.4%, and 13.7%, respectively.

The literature lacks in-depth analysis and identification of practices for implementing DevOps in software organizations. The purpose of the current study is to identify practices that are heading toward DevOps. Interviews with industry experts in DevOps are considered in the research, revealing various practices that have not been mentioned in other studies. The next step is to prioritize the discovered practices using the FBWM technique, a new approach to MCDM that combines fuzzy theory to account for input uncertainty and yield more accurate outcomes. The current study suggests several innovative software practices to address earlier works' shortcomings.

3 | RESEARCH DESIGN

The study was conducted using a three-step research design to accomplish the goals and answer to the research as mentioned above in research questions: (1) Conducting an SLR to identify software practices for implementing DevOps, (2) validating the identified software practices through a questionnaire study with the experts, (3) applying the best–worst method (BWM) to prioritize and develop a taxonomy of DevOps practices by aggregating the responses of DevOps experts participated in empirical validation. The stages of the research methodology are summarized in Figure 1.

3.1 | SLR

An SLR is a secondary study that locates, chooses, and assesses existing research to answer clearly defined questions.³⁵ SLR provides a foundation or background for carrying out additional new research initiatives. Most investigations begin with a literature review. As a result, the SLR must be conducted impartially using a predetermined search strategy. We have used Kitchenham and Charters^{36,37} predefined technique to run the SLR in this study. As discussed in Table 1, the SLR of Kitchenham³⁶ and Charters³⁷ comprises three primary phases: preparing the review, carrying it out, and reporting it.

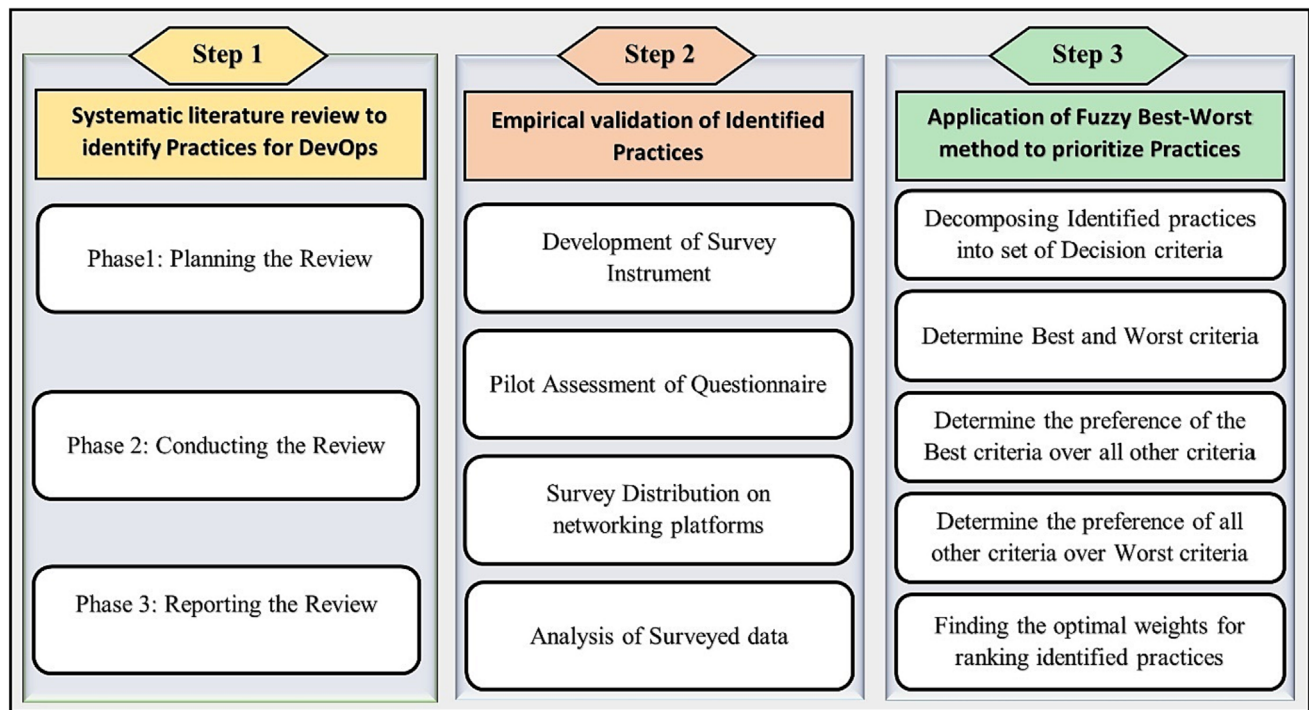


FIGURE 1 Research design.

TABLE 1 SLR.

Phases	Tasks
Planning review	Research questions Data source selection Search keywords Criteria for exclusion and inclusion Criteria for quality assessment
Executing review	Primary studies selection Data extraction and synthesis
Reporting review	Report findings

3.1.1 | Planning review

Research questions

The purpose of SLR in the present study is to extract software practices that help DevOps adoption in software organizations. The research questions formulated for this study have already been covered in the discussion above in Section 1.

Data sources selection

Several digital libraries, electronic databases, or online repositories offer a wealth of knowledge on various subjects. Based on our research knowledge and Chen's recommendations,³⁸ we have selected six digital libraries to gather relevant data for this study.

1. "ACM Digital Library (<https://dl.acm.org>)"
2. "IEEE Xplore (<https://ieeexplore.ieee.org>)"
3. "Wiley Online Library (<https://onlinelibrary.wiley.com>)"
4. "Springer Link (<https://link.springer.com>)"
5. "Science Direct (<https://www.sciencedirect.com>)"
6. "Google Scholar (<https://scholar.google.com>)"

Search process

The search phrase was created by combining terms from our research questions and taking knowledge from existing DevOps literature.^{9,18,22,23,25} The two Boolean operators "AND" and "OR" are used for joining various keywords to create search strings. The "AND" operator connects different words, whereas the "OR" operator joins two words with the same meaning. The search string prepared for this study is as follows:

("Practices" OR "Dimensions" OR "Principles" OR "Capabilities" OR "Perspectives" OR "Areas") AND ("DevOps" OR "Continuous Integration" OR "Continuous Deployment" OR "Continuous Delivery") AND ("capability" OR "adoption" OR "maturity") AND ("software organizations" OR "software companies" OR "IT companies" OR "software industry" OR "IT industry" OR "IT enterprises")

Criteria for inclusion and exclusion

After running a search query, the inclusion and exclusion criteria were created to clean up the content obtained from various databases. The information retrieved from several databases may have contained some papers that matched the search query but were irrelevant to our study. IC and EC were created by considering research done in the DevOps discipline.^{39–43}

Inclusion criteria (IC)

1. Peer-reviewed published research (i.e., journal articles, conference proceedings, and symposium/workshop papers).
2. The research studies should be expressed in English.
3. Research studies whose full-text versions are available.
4. Research studies that focus on DevOps in the software engineering domain.

Exclusion criteria (EC)

1. Duplicate articles are excluded.
2. Research studies of gray literature are excluded.
3. The research studies with the maximum contribution should be considered if multiple studies are published in the same project.
4. Articles that use DevOps as a background are neglected.

Quality assessment (QA) criteria

The selected article undergoes QA criteria with the data extraction phase after they are fetched from databases. The criteria are to select the best articles for our primary study. The quality article will help us find better results in our research. The questions for QA are based on guidelines In.^{44–46} The list (Table 2) contains the following quality criteria assessment questions:

TABLE 2 Quality assessment criteria.

S. No.	Questions checklist	Evaluation score
QA1	Is the used research methodology described clearly?	(No = 0; Yes = 1; Partial = 0.5)
QA2	Are the objectives of the research study clearly defined?	(No = 0; Yes = 1; Partial = 0.5)
QA3	Are the experimental parameters stated explicitly?	(No = 0; Yes = 1; Partial = 0.5)
QA4	Are the study's implications in the actual world reported?	(No = 0; Yes = 1; Partial = 0.5)

3.1.2 | Executing the review

Primary studies selection

The tollgate approach suggested by Afzal et al⁴⁷ was applied to identify the most relevant articles for the primary study. This strategy helps to reduce the enormous volume of publications with the application of these five steps:

Phase 1 (Ph-1): Applying the search query to explore relevant articles on digital sources.

Phase 2 (Ph-2): Applying inclusion/exclusion criteria.

Phase 3 (Ph-3): Including articles through reading titles and abstracts.

Phase 4 (Ph-4): Including articles through reading the introduction and conclusion.

Phase 5 (Ph-5): Including articles through the reading of the full text.

In the first phase, a search string was used to discover pertinent articles across six digital libraries: Google Scholar, IEEE Xplore, Springer Link, Wiley Inter Science, and ACM Digital Library. By using the search string outlined in Section 0, a total of 781 articles were obtained. In the second phase, inclusion and exclusion criteria were applied to the research articles retrieved from different databases, resulting in 372 articles. Subsequently, in the third phase, the relevance of the documents was determined by examining the title and abstract, yielding 202 studies. In the fourth phase, the introduction and conclusion were scrutinized, leading to 108 studies. Finally, in the fifth phase, the entire text of the articles was read, resulting in a final selection of 68 studies for the primary investigation. Table 3 and Figure 2 provide a summary of the remaining articles after completing all five phases of the tollgate approach.

Data extraction and synthesis

This section defines the data extraction process that will be used to determine whether the findings of the selected studies were relevant to our research problem. The articles used for the primary study were carefully reviewed to identify practices, approaches, concepts, and contributions to DevOps implementation. To reduce the subjectivity of the data extraction and synthesis process, at least two authors had to agree on each outcome; the authors independently produced their identifications, and when there were discrepancies, each paper was individually reviewed until a consensus was obtained. To gather the software practices mentioned in each of the 68 primary studies, we examined them for this study. Furthermore, even though there were no strict regulations for the extraction procedure, we followed the basic rules:

- Firstly, searched by the term “practice” for implementing DevOps in the selected studies.
- If not mentioned in the form of practices, then searched by other names such as capabilities, principles, dimensions, and areas.
- Retained all those practices, capabilities, dimensions, fields, areas, or principles mentioned and explained in the texts.

The software practices have been carefully compiled to keep the list manageable. The identified software practices for implementing DevOps from primary studies are summarized. The first and second authors were involved in extracting software practices. The purpose of the third author was to carefully review all practices identified from the selected primary article to eliminate research bias. Because numerous authors were involved in the data validation, this strategy's application increased the validity.

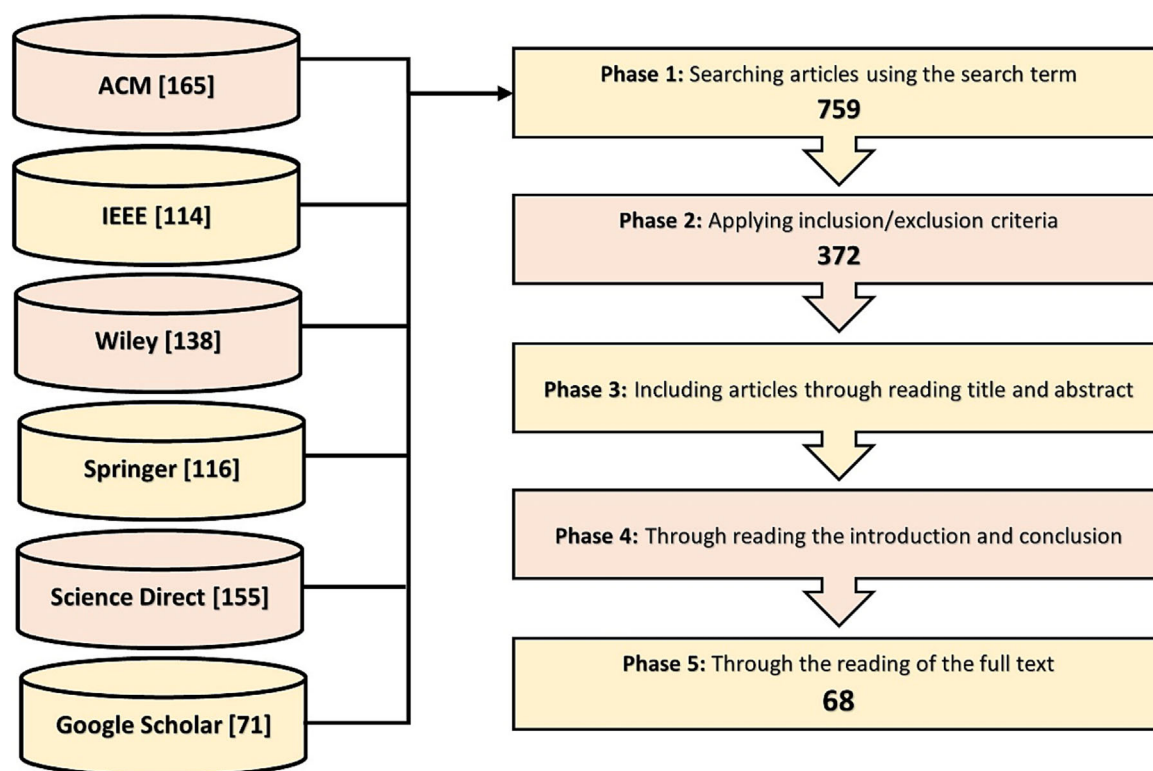
3.1.3 | Reporting review

Distribution of studies over the years

The temporal distribution of DevOps-related articles published over the years is analyzed in this section. According to the articles, DevOps has acquired attention and popularity and is predicted to keep expanding. A slight increase in the number of documents has been observed since 2016 and a rapid rise in interest after 2017. Figure 3 depicts the distribution of articles linked to DevOps. This illustrates the growth in research activity over the last 3 years relating to the techniques, areas, and capabilities for adopting DevOps, indicating the potential interest and use of this study field.

TABLE 3 Tollgate approach.

E-resources	Ph1	Ph2	Ph3	Ph4	Ph5 (Final selection)	%Age
ACM Digital Library	165	74	44	19	12	18
IEEE Xplore	114	86	34	17	12	18
Wiley Online Library	138	69	37	15	10	14
Springer Link	116	44	29	21	13	19
Science Direct	155	60	31	14	8	12
Google Scholar	71	39	27	22	13	19
Total	759	372	202	108	68	100%

**FIGURE 2** Pictorial representation of the tollgate approach.

QA of studies

The quality of the primary studies was assessed with the help of four criteria designed for QA (discussed in Table 2). A value of 0 or 0.5, or 1 was assigned to studies for each of the four criteria. The sum of the scores for each QA question was used to generate the final score for each study. The aggregate QA question score indicates that all the chosen studies scored 50% or higher, proving the relevance of the primary study in answering research questions.

Research design in studies

The selected primary studies used a variety of research methods in their studies. We have analyzed the research methodology adopted in each primary selected study. We found various types of studies, including 14 studies case studies (CS), 16 mixed methods (MM), 4 based on grounded theory (GT), 8 content analysis (CA), 13 based on SLR, 3 multivocal literature review (MVLr), 3 design science (DS), and 7 questionnaire survey studies (QS). The research methods used in various studies of our selected articles are CS (21%), MM (24%), GT (6%), DS (4%), CA (12%), SLR (19%), MVLr (4%), and QS (10%). Figure 4 depicts the percentage of each research method used in selected articles for our primary study.

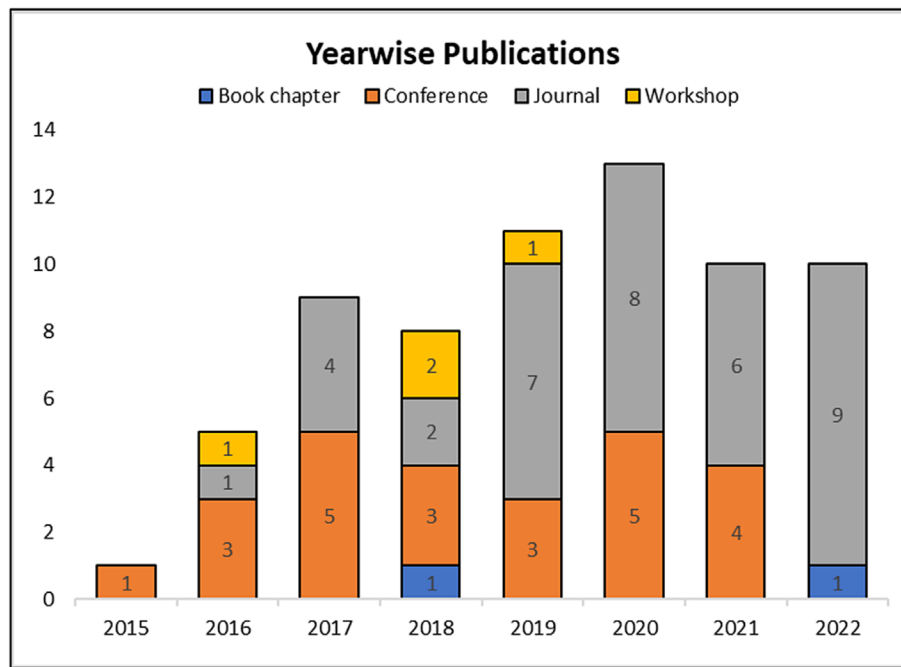


FIGURE 3 Yearly distribution of selected primary studies.

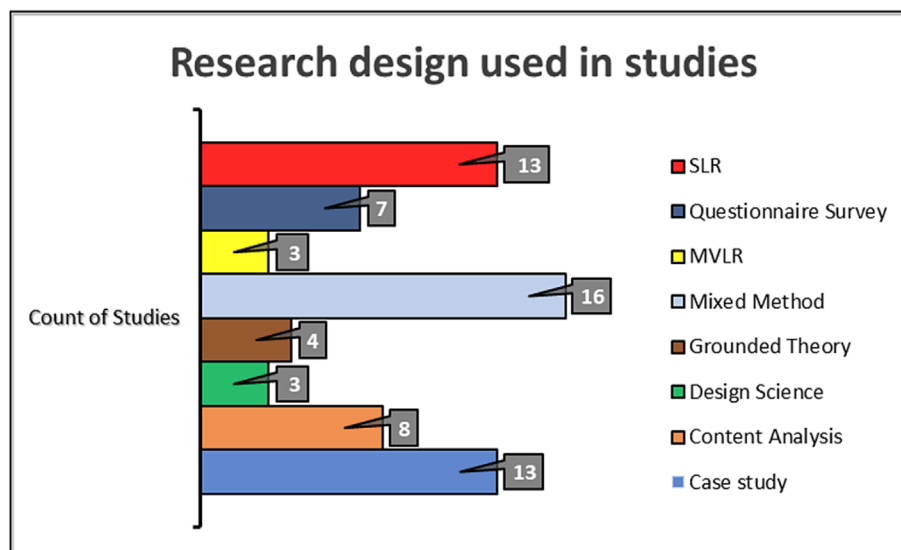


FIGURE 4 Research design adopted in selected primary studies.

3.2 | Empirical validation of identified software practices

We have undertaken an empirical study through a questionnaire survey to validate the practices identified from SLR. Moreover, the empirical study was also crucial in identifying new practices in the real-world environment not identified in the literature. The survey method used for the empirical study was designed by considering the guidelines suggested by Linåker et al.⁴⁸ and Groves et al.⁴⁹

3.2.1 | Design the survey instrument

A questionnaire survey is a practical method for obtaining the most important information from the scattered and targeted population.⁵⁰ We have designed and executed an online questionnaire survey to validate the DevOps best practices identified in the SLR study. We have used Google

Forms (docs.google.com/forms) for creating the instrument. The questionnaire survey was divided into two main components, which are as follows: (1) In the first section, questions were asked about the demographic details of the participants, including their job title, experience, nationality, and organizational characteristics like size, type of scope, and employee count. (2) A list of identified DevOps best practices is presented. The survey participants were asked to mention the importance of each software practice on a five-point Likert scale: strongly disagree, disagree, neutral, agree, and strongly agree.

3.2.2 | Pilot assessment of the questionnaire

The pilot evaluation of the questionnaire is essential to find the possible weaknesses related to how the DevOps practitioners easily understand the questions.⁵¹ We invited three external experts to conduct a pilot evaluation of the developed questionnaire. One of the experts is academic, and two industry professionals have more than 5 years of research experience in DevOps and empirical software engineering. We have provided the questionnaire to every expert via email and asked them for suggestions to improve the quality of the questionnaire. We received several suggestions related to the format of the questionnaire. Moreover, one of the experts suggested including one question related to the bibliographic information of survey participants. The questionnaire was revised after carefully considering the expert comments, and the final questionnaire is prepared in Appendix 2.

To evaluate the interpersonal biases, Kendall's coefficient of concordance (W) test has been chosen.^{52,53} Three external reviewers are invited to evaluate the interpersonal biases in the review process. W is a standard statistical approach to assess the level of agreement between a group of people that evaluate a set of n objects⁵⁴ [98]. The value of w lies between 0 and 1, where $W = 0$ shows a complete disagreement level between the people and $W = 1$ refers to complete agreement⁵⁴ [98]. In this study, the code for W test is developed using R-3.6.3 and calculated the W value to assess the external reviewers' level of agreement during the pilot study. The results given in Table 4 ($W = 0.833$) show that the invited external reviewers are at a positive agreement level for the questionnaire survey. The following is the R code for W :

```
library(DescTools)
Agile<-data.frame(
external_rev1 = c(7,4,6,2,3,9,8,5,10,12,18,13,15,19,11,14,16,3,4)
external_rev2 = c(6,7,5,2,3,8,10,9,5,3,11,14,13,15,12,17,18,16,15)
authors = c(8,3,7,2,4,3,15,9,8,6,14,10,12,11,13,6, 8, 13, 15))
KendallIW (Agile, TRUE)
)
KendallIW (Agile, TRUE, test = TRUE)
)
KendallIW(t(d.att[, -1]), test = TRUE)
```

3.2.3 | Survey distribution

To collect the responses, we approached the DevOps practitioners through the different groups hosted by LinkedIn and ResearchGate. Initially, we requested to be part of the groups and then posted our questionnaire in the groups. Moreover, we also sent emails to GitHub users asking them to complete the survey. We collected 110 responses from the survey participants. All the responses were manually checked by all authors and found three incomplete responses. After excluding the incomplete responses, we have retained 107 responses for data analysis.

3.2.4 | Data analysis

Data gathered from Likert-scale questions were analyzed using frequency analysis, a valuable method for analyzing quantitative and qualitative data. In our study, we considered frequency analysis as the best approach to analyze the opinions across the variables and groups of variables. Many researchers in different areas of software engineering have used the same strategy in various domains.

TABLE 4 Inter-reliability test.

Data set	Kendall chi-square	df	Subjects	Raters	p	W
SAD	68.5132	18	19	3	0.001	0.833

3.3 | Fuzzy best–worst MCDM method

In this study, the FBWM was applied to determine priority weights and develop the taxonomy of the practices identified for DevOps from an SLR by aggregating responses of DevOps experts who participated in empirical validation. The actual implementation of FBWM has been elaborated in Section 4.3. This section describes a novel MCDM model built on the FBWM and its driving concepts in detail as follows.

3.3.1 | Fuzzy set concept

A concept representing fuzziness as a quantitative number is denoted as a fuzzy set.^{55,56} Thus, a fuzzy set is usually denoted by a membership function that can have any value of membership between complete membership (1) and non-membership.⁵⁷

A fuzzy set's mathematical interpretation can be illustrated as follows:

$$\tilde{S} = \{(x, \mu_F(x)) | x \in X\}.$$

Here, the membership function $\mu_F(x)$ maps an item between 0 and 1.

A triangular fuzzy number (TFN)⁵⁶ is represented by a triplet: $\tilde{S} = (l, m, u)$, and

The following equations define the membership function:

$$\mu_F(x) = \begin{cases} \frac{x-l}{m-l} & \text{for } l \leq x \leq m \\ \frac{u-x}{u-m} & \text{for } m \leq x \leq u \\ 0, & \text{Otherwise} \end{cases}$$

where the lower, middle, and upper values of a crisp number are represented by l , m , and u , respectively.

Decision-makers generally make fuzzy pairwise comparisons between two criteria with the help of linguistic expressions ranging from “equally important” to “extremely important.” Then, for applying FBWM, it is necessary to transform these linguistic terms into fuzzy triangular numbers based on the rules listed in Table 5 and demonstrated in Figure 5.

3.3.2 | BWM

The BWM was introduced by Rezaei.⁵⁸ In contrast to AHP, which requires $n(n-1)/2$ pairwise comparisons, BWM only needs $2n-3$ pairwise comparisons ($n-2$ best-to-others + $n-2$ others-to-worst [OW] + 1 best-to-worst comparison).⁵⁹ Because BWM involves fewer pairwise comparisons than other MCDM techniques, it is preferable when there are many criteria to consider. In the BWM, all other criteria are compared to the worst criterion, whereas the best criterion is compared to the other left criteria. In this method, the decision-maker is asked to indicate how they would prefer the best criterion to all other criteria and how they would prefer all other criteria to the worst criterion. The best-to-others

TABLE 5 Transformation scheme for representing linguistic terms into TFN.

Crisp numbers	Linguistic terms	Triangular fuzzy number
9	Extremely important	(4.25, 4.5, 4.75)
8	Intermediate	(3.75, 4, 4.25)
7	Very important	(3.25, 3.5, 3.75)
6	Intermediate	(2.75, 3, 3.25)
5	Fairly important	(2.25, 2.5, 2.75)
4	Intermediate	(1.75, 2, 2.25)
3	Weakly important	(1.25, 1.5, 1.75)
2	Intermediate	(0.75, 1, 1.25)
1	Equally important	(1, 1, 1)

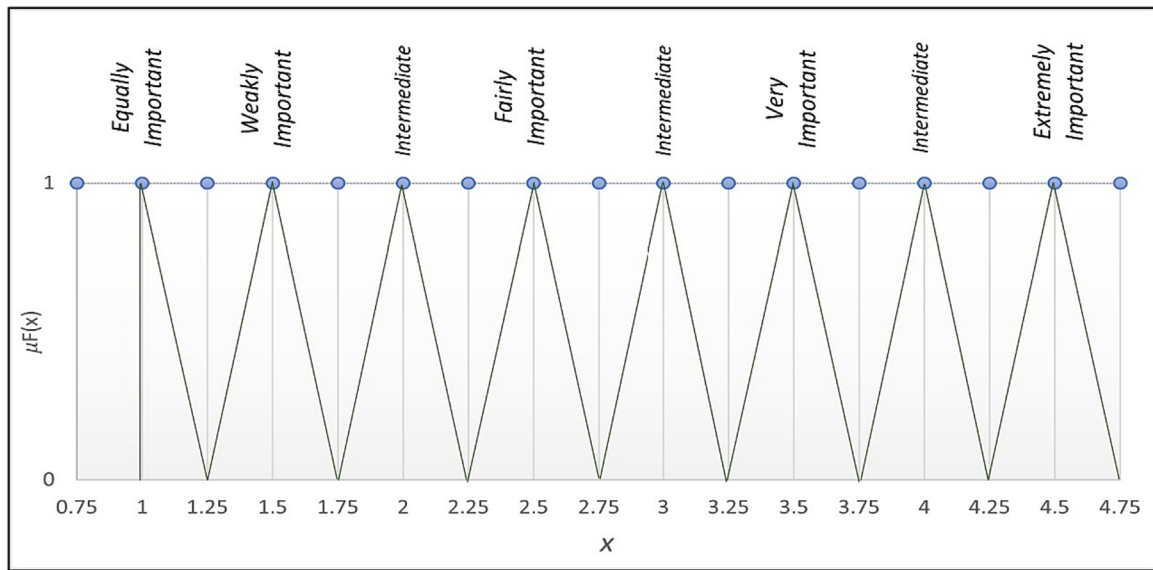


FIGURE 5 Triangular fuzzy numbers representation.

(BO) vector and others-to-best vector, produced by these pairwise comparisons, are used to formulate a max-min optimization problem that eventually determines the weight of each criterion and the consistency ratio for checking the validity of comparisons.

3.3.3 | FBWM

This section overviews Guo and Zhao's fuzzy best-worst technique.⁶⁰ The goal of FBWM is to calculate fuzzy weights of multiple criteria and subcriteria.⁶¹ We opted for the FBWM over other MCDM techniques for the following reasons.

(a) Reduction in pairwise comparisons: FBWM requires fewer pairwise comparisons as compared to other MCDM methods.⁶² In FBWM, only reference comparisons are considered. Reference comparison refers to determining the preference of the best criterion over all other criteria and the preference of all criteria over the worst criterion by rating it on a scale of 1 to 9. The remaining comparisons that are not considered in BWM fall under secondary comparisons. This can be advantageous when there are many criteria to consider in the decision-making process. With a reduced number of pairwise comparisons, the decision-making process becomes less time-consuming and complex.

(b) Simplicity: BWM is a relatively straightforward method that is easy to understand and implement.⁶³ It provides a simple framework for decision-makers to evaluate alternatives based on their relative importance with respect to the best and worst criteria.

(c) Fuzzy logic integration: Due to the ambiguity and uncertainty inherent in human qualitative judgments (giving preference in linguistic terms), combining conventional BWM with fuzzy set theory is suitable for understanding decision-maker judgments better.⁶⁴ Therefore, the reference comparisons of BWM can be performed using fuzzy triangular numbers in place of linguistic terms that appear more consistent with the actual scenario and produce more concrete results.

The following explains FBWM's specific steps. To calculate the weights of both criterion and subcriteria, these procedures must be followed. The final fuzzy weights of the subcriteria are obtained by multiplying the fuzzy weights of the subcriteria by the relevant criterion.

1. Build the relative set of decision criteria.

A system of decision criteria includes a number of relative criteria that are essential for selecting among subcriteria. The set of decision criteria is represented as

$$\{C_1, C_2, \dots, C_n\}.$$

2. Determine the best and worst criteria from a set of decision criteria.

From the system of decision criteria created in Step 1, experts can pick the best and worst criterion. Then based on the selection of the experts, the best criterion, C_B , and the worst criterion, C_W , was identified.

3. Obtain the preference of the best criterion over all other criteria.

Using the linguistic values presented in Table 5, the experts are asked to express their preferences for the best criterion relative to all other criteria. Then the obtained fuzzy preferences are transformed into fuzzy triangular numbers. Experts' choice for the best criterion compared to other criteria is written as a fuzzy BO vector $A_B = (a_{B1}, a_{B2}, \dots, a_{Bn})$, where a_{Bj} represents the fuzzy preference of best criteria C_B over criteria $C_j, j = 1, 2, \dots, n$ and $a_{BB} = (1, 1, 1)$.

4. Obtain the preference of all other criteria over the worst criteria.

The process is similar to the previous step 3. But here, the experts are asked to state their preferences for all other criteria against the worst criteria, using the linguistic terms listed in Table 5. Then, the obtained fuzzy preferences are transformed into TFNs. Experts' preference for all other criteria in comparison to worst criteria is written as a fuzzy OW vector $A_W = (a_{1W}, a_{2W}, \dots, a_{nW})$, where a_{jW} indicates the fuzzy preference of criteria $C_j, j = 1, 2, \dots, n$ over the worst criteria C_W and $a_{WW} = (1, 1, 1)$.

5. Calculate the fuzzy weights $(w_1^*, w_2^*, \dots, w_n^*)$ of each of the criteria.

To obtain optimal weights, it should follow. $\frac{w_B}{w_j} = a_{Bj}$ and $\frac{w_j}{w_W} = a_{jW}$, which is satisfied when the absolute gaps $\left| \frac{w_B}{w_j} - a_{Bj} \right|$ and $\left| \frac{w_j}{w_W} - a_{jW} \right|$, for all j are minimized.

To calculate the fuzzy weights, the following constrained optimization problem is put forth:

$$\min \max_j \left\{ \left| \frac{w_B}{w_j} - a_{Bj} \right|, \left| \frac{w_j}{w_W} - a_{jW} \right| \right\},$$

$$\text{s.t.} \begin{cases} \sum_{j=1}^n R(w_j) = 1 \\ l_j^w \leq m_j^w \leq u_j^w, \\ l_j^w \geq 0 \\ j = 1, 2, \dots, n \end{cases}$$

where $w_B = (l_B^w, m_B^w, u_B^w)$, $w_j = (l_j^w, m_j^w, u_j^w)$, and $w_W = (l_W^w, m_W^w, u_W^w)$.

$a_{Bj} = (l_{Bj}, m_{Bj}, u_{Bj})$ and $a_{jW} = (l_{jW}, m_{jW}, u_{jW})$ are all fuzzy triangular numbers.

The following non-linear constrained optimization problem may be formed from this equation.

$$\min \xi$$

$$\text{s.t.} \begin{cases} \left| \frac{w_B}{w_j} - a_{Bj} \right| \leq \xi \\ \left| \frac{w_j}{w_W} - a_{jW} \right| \leq \xi \\ \sum_{j=1}^n R(w_j) = 1 \\ l_j^w \leq m_j^w \leq u_j^w \\ l_j^w \geq 0 \\ j = 1, 2, \dots, n \end{cases}$$

In the above equation, when the values of w_B, w_j, w_W and a_{Bj}, a_{jW} are replaced with the fuzzy triangular number representation as below:

w_B = fuzzy weights of best criteria, written as $w_B = (l_B^w, m_B^w, u_B^w)$,

w_j = fuzzy weights of j^{th} criteria, written as $w_j = (l_j^w, m_j^w, u_j^w)$,

w_W = fuzzy weights of worst criteria, written as $w_W = (l_W^w, m_W^w, u_W^w)$

a_{Bj} = preference of best criteria with respect to j^{th} criteria, written as $a_{Bj} = (l_{Bj}, m_{Bj}, u_{Bj})$

a_{jW} = preference of j^{th} criteria with respect to worst criteria, written as $a_{jW} = (l_{jW}, m_{jW}, u_{jW})$

and also, when ξ represented in TFN, it becomes $\xi = (l^\xi, m^\xi, u^\xi)$.

Being considering $l^\xi \leq m^\xi \leq u^\xi$, we supposed $\xi^* = (k^*, k^*, k^*), k^* \leq l^\xi$.

As a result, the equation will become

$$\begin{aligned} & \min_{\xi^*} \\ & \text{s.t.} \left\{ \begin{array}{l} \left| \frac{(l_B^w, m_B^w, u_B^w)}{(l_j^w, m_j^w, u_j^w)} - (l_{Bj}, m_{Bj}, u_{Bj}) \right| \leq (k^*, k^*, k^*) \\ \left| \frac{(l_j^w, m_j^w, u_j^w)}{(l_W^w, m_W^w, u_W^w)} - (l_{jW}, m_{jW}, u_{jW}) \right| \leq (k^*, k^*, k^*) \\ \sum_{j=1}^n R(w_j) = 1 \\ l_j^w \leq m_j^w \leq u_j^w \\ l_j^w \geq 0 \\ j = 1, 2, \dots, n \end{array} \right. \end{aligned}$$

By solving the above equation, the optimal fuzzy weights $(w_1^*, w_2^*, \dots, w_n^*)$ of each criterion and the optimal value of ξ^* are obtained.

The consistency ratio is determined by dividing ξ^* values for comparing the best criterion to the worst criterion with their corresponding consistency index mentioned in Table 6. It implies that the comparison scheme given by the decision-makers is more consistent when the value of ξ^* is closer to zero.⁶⁵ To ensure the consistency of the comparisons:

$$\text{Consistency Ratio} = \frac{\xi^*}{\text{Consistency Index}}.$$

4 | RESULTS AND ANALYSIS

The following sections cover the findings of the SLR, the empirical investigation, and the FBWM.

4.1 | Findings of SLR

The SLR effectively addressed the first research question by revealing a list of 19 software practices. The practice list provided in Table 7 is derived from a comprehensive survey of 68 publications. It should be noted that practitioners may also refer to these practices by different names, such as capabilities, areas, dimensions, or principles. Within the literature, it is acknowledged that these practices have the potential to bring various advantages to different aspects of DevOps. The identified practices encompass different stages of the software development life-cycle, ensuring comprehensive coverage. For instance, in the requirements gathering and planning phase,⁶⁶ practices like P1: Architecture Specifications, P3: Backlog Management, and P19: User Stories-based Implementation are essential.

Similarly, in the testing and quality assurance phase,⁶⁷ P2: Automated Unit Testing and P13: End-to-End System Testing play crucial roles. Regarding code development and maintenance,⁶⁸ practices like P5: Code Review, P6: Coding Standards, P7: Collective Code Ownership, P10: Daily Standup, and P16: Code Refactoring are important. These practices focus on improving the efficiency and effectiveness of code-related

TABLE 6 Consistency index for fuzzy BWM.

a_{BW}	(1, 1, 1)	(0.75, 1, 1.25)	(1.25, 1.5, 1.75)	(1.75, 2, 2.25)	(2.25, 2.5, 2.75)	(2.75, 3, 3.25)	(3.25, 3.5, 3.75)	(3.75, 4, 4.25)	(4.25, 4.5, 4.75)
Consistency index	3.00	3.80		5.29		6.69		8.04	

TABLE 7 Results of the empirical study.

S. No.	Practices for DevOps	Empirical observations (N = 107)							
		Positive			Negative			Neutral	
		SA	A	%	SD	D	%	N	%
P1	Architecture Specifications	20	61	76	5	12	16	9	8
P2	Automated Unit Testing	22	67	83	3	9	11	6	6
P3	Backlog Management	18	61	74	7	13	19	8	7
P4	Burndown Charts	19	59	73	6	14	19	9	8
P5	Code Review	22	67	83	5	9	13	4	4
P6	Coding Standards	23	69	86	3	9	11	3	3
P7	Collective Code Ownership	16	65	76	4	12	15	10	9
P8	Automated Deployment Pipeline	28	69	91	2	4	6	4	4
P9	Automated Integration of Code Changes	27	70	91	0	5	5	5	5
P10	Daily Standup	16	66	77	6	7	12	12	11
P11	Defining the Deployed Features	15	62	72	4	15	18	11	10
P12	Estimating Team Effort	17	65	77	3	11	13	11	10
P13	End-to-End System Testing	18	61	74	5	9	13	14	13
P14	Iteration/Sprint Reviews	14	67	76	3	16	18	7	7
P15	Limit work in Progress	14	62	71	5	13	17	13	12
P16	Code Refactoring	22	66	82	3	7	9	9	8
P17	Planning for Early Release	23	66	83	2	9	10	7	7
P18	Effective Sprint Retrospectives	22	65	81	3	8	10	9	8
P19	User Stories-based Implementation	23	69	86	3	6	8	6	6

activities. For deployment and delivery,⁶⁹ practices like P8: Automated Deployment Pipeline and P11: Defining the Deployed Features are crucial. They streamline the software deployment process and ensure the correct features are released. Lastly, practices such as P4: Burndown Charts, P14: Iteration/Sprint Reviews, P15: Limit work in Progress, and P18: Effective Sprint Retrospectives contribute to continuous improvement⁷⁰ in the DevOps lifecycle. These practices enable teams to analyze and enhance their processes iteratively.

To facilitate understanding, these practices can be grouped into different phases of the DevOps lifecycle, namely, Continuous Planning,^{71,72} Continuous Development,⁷³ Continuous Testing,^{74,75} Continuous Integration,⁹ Continuous Deployment,⁷⁶ and Continuous Monitoring.⁷⁷ These groupings align with industry standards and reflect the interconnected nature of these practices in achieving successful DevOps implementations. The grouping of practices has been discussed in a later section.

4.2 | Findings of empirical study

4.2.1 | Demographic details of participants

For a visual representation of the demographic information, please refer to the accompanying Figure 6. Regarding the demographic details of the organizations surveyed, it was found that 65% of the organizations had a team size exceeding 50 professionals. This indicates that a significant portion of the surveyed organizations had large teams working on DevOps practices. On the other hand, 9% of the organizations had fewer than 10 professionals, suggesting that a small fraction of the surveyed organizations had relatively small teams. The remaining 26% of organizations fell into the middle range, with varying numbers of professionals ranging from 20 to 50. This finding implies that a moderate-sized team was present in a considerable portion of the surveyed organizations.

When examining the experience of the DevOps professionals who responded to the survey, it was observed that the majority, accounting for 36% of the respondents, had a working experience in the 6–10-year range. This suggests that a significant proportion of the surveyed professionals had a substantial amount of experience in implementing DevOps practices. The second most common range of working experience was 1–5 years, which accounted for 28% of the respondents. This indicates that a considerable number of professionals with relatively less experience were also part of the survey. The third, fourth, and lowest ranges of working experience were 11–15 years (18%), less than 1 year (12%), and

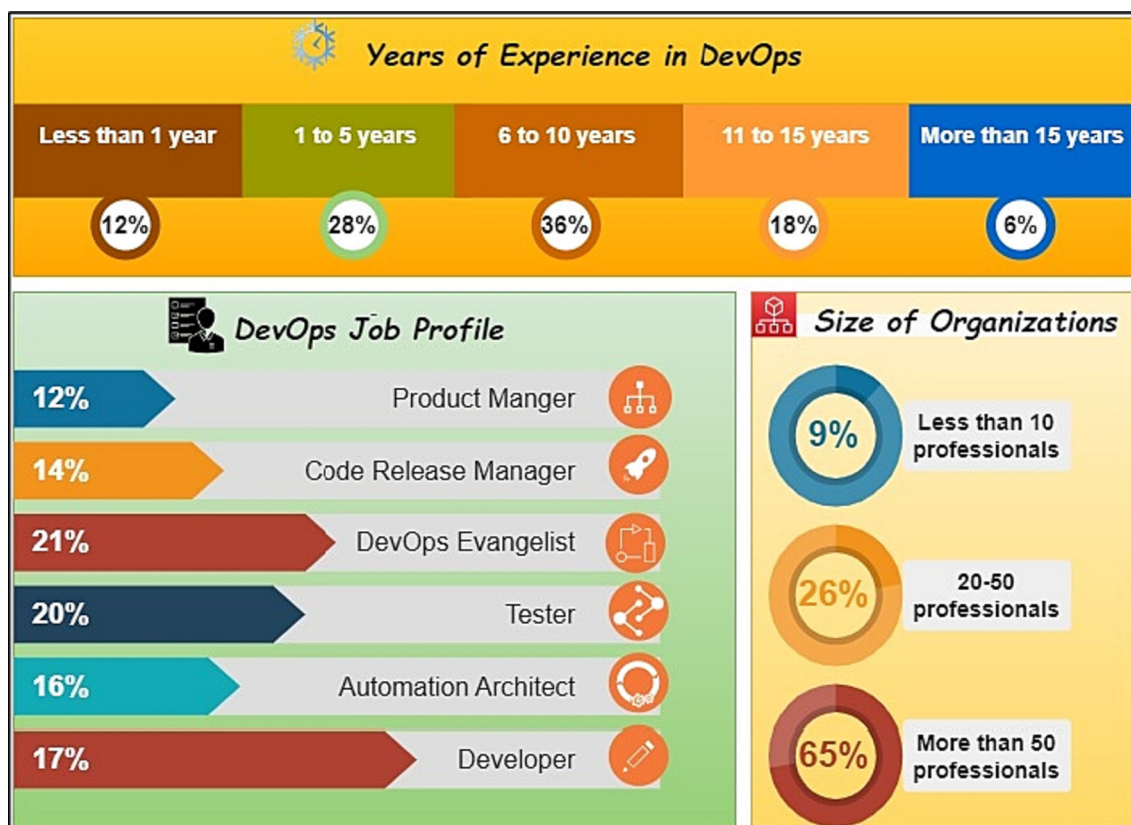


FIGURE 6 Demographics of survey respondents.

more than 15 years (6%), respectively. These findings reveal that smaller percentages of respondents had experience levels beyond the 6–10-year range, with the smallest proportion having over 15 years of experience.

The survey results revealed a diverse range of job titles among the respondents. The most prevalent job title was Automation Architect, accounting for 16% of the participants. These professionals specialize in designing and implementing automated systems to streamline processes and improve efficiency. Testers, constituting 20% of the respondents, play a critical role in ensuring software quality by conducting rigorous testing and identifying bugs and issues. Developers, comprising 17% of the participants, are responsible for creating and maintaining software applications, employing various programming languages and frameworks. The survey also highlighted the prominence of DevOps evangelists, making up 21% of the respondents. These individuals advocate for the adoption of DevOps practices, emphasizing collaboration, automation, and continuous delivery. Code Release Managers, comprising 14% of the participants, oversee the deployment of software releases, coordinating with development and operations teams to ensure smooth and efficient releases. Lastly, Product Managers accounted for 12% of the respondents, indicating the significance of their role in defining product vision, strategy, and roadmap to meet customer needs.

4.2.2 | Validation responses

In response to the questionnaire, 110 participants responded. All authors manually checked all responses and noticed three incomplete responses. We kept 107 responses after removing the incomplete responses for data analysis. Survey respondents' positive, negative, and neutral responses were divided into three groups (Table 7). The percentage of survey participants who believed the identified factors had a good influence on DevOps practices is shown in the positive ("strongly agree and agree") category. The respondents who did not think that the reported practice had any impact on the adoption of DevOps are shown in the negative (strongly disagree, disagree) category. In the neutral group, the results of respondents who were unclear about the importance and influence of software practice are displayed (Table 7). Automated Deployment Pipeline (P8) and Automated Integration of Code Changes (P9) were considered to be the most significant (91%) practices according to the survey respondents. Automated Deployment Pipeline and Automated Integration of Code Changes are two key DevOps implementation practices that can benefit software development teams significantly. These practices help to improve collaboration between teams, accelerate the development process, reduce risk, and provide faster feedback on the quality of the code changes. Similarly, the results displayed in Table 7 show that 86% of

the survey respondents identified Coding Standards (P6) and User Stories–based Implementation (P19) as the second most common practices for implementing DevOps. Coding Standards and User Stories–based implementation can help development teams build high-quality software that meets users' needs. By following coding standards, teams can produce easy-to-maintain and enhanced code, reducing the likelihood of bugs and errors. User Stories–based implementation ensures that the software being developed meets users' needs, which can lead to higher user adoption rates and increased customer satisfaction. These practices also help ensure that software development teams are more productive and efficient, which is essential for successful DevOps implementation.

4.3 | Findings of FBWM implementation

FBMW has been used to determine the importance weighting of software development practices for DevOps implementation. The following is a description of the steps that BWM takes to determine the weight of the criteria:

1. **Determining the set of Decision criteria $\{C_1, C_2, \dots, C_n\}$ based on the extensive literature review:** The software practices determined from the SLR were used as decision criteria. There were identified software practices, 19 that must be used as criteria for applying to FBWM. But FBWM's developers have recommended grouping the criteria into smaller groups of more than nine criteria. Doing so raises the problem's hierarchy by one level.⁷⁸ To do so, we use conceptual mapping to map the identified practices into the six stages of the DevOps lifecycle to execute the clustering. To ensure a comprehensive and accurate mapping, all authors of the study participated in the process. To do the categorization of the given practices, we analyzed each practice and determined which category it aligns with based on its characteristics and objectives. The categorization of the given practices can be justified based on their respective characteristics and roles within the software development process. Here is a breakdown of each activity and its corresponding category:

(a) Continuous Planning (C1):

- Backlog Management (P3): Backlog management is an essential part of continuous planning.⁷⁹ It involves prioritizing and managing the backlog of user stories or tasks. This activity falls under continuous planning as it focuses on the ongoing planning and refinement of the project backlog.
- Daily Standup (P10): The Daily Standup is part of Continuous Planning because it provides a regular opportunity for the team to plan their day and align their efforts.⁸⁰ It helps in identifying any obstacles or dependencies that may affect the planned work and allows for course correction if needed.
- Effective Sprint Retrospectives (P18): Sprint retrospectives are part of the agile methodology and occur at the end of each sprint or iteration.⁸¹ They provide an opportunity for the team to reflect on their performance, identify areas of improvement, and make necessary adjustments. This activity aligns with continuous planning as it supports iterative planning and improvement.
- User Stories–based Implementation (P19): User stories are a key aspect of agile development, enabling the team to focus on user requirements and deliver incremental value.⁸² This activity involves planning and implementing user stories throughout the development process.

(b) Continuous Development (C2):

- Code Review (P5): Code review is an integral part of continuous development.⁸³ It involves reviewing code changes to ensure quality, maintainability, and adherence to coding standards. Code review helps identify and fix issues early in the development process.
- Coding Standards (P6): Coding standards are guidelines that define a consistent style and structure for writing code.⁸⁴ They promote maintainability, readability, and collaboration within the development team. Adhering to coding standards is a fundamental practice in continuous development.
- Collective Code Ownership (P7): Collective code ownership emphasizes that the entire development team is responsible for the quality and maintenance of the codebase.⁸⁵ It encourages collaboration, knowledge sharing, and continuous improvement within the team.

(c) Continuous Testing (C3):

- Automated Unit Testing (P2): Automated unit testing is a practice where individual units of code are tested in isolation to ensure they function correctly.⁸⁶ It is an integral part of continuous testing, as it helps catch issues early and ensures code quality and reliability.
- Estimating Team Effort (P12): Estimating Team Effort is part of Continuous Testing as it involves evaluating the complexity and effort required to implement and test a particular feature.⁸⁷ It helps in planning and allocating resources effectively, including testing efforts.

- End-to-End System Testing (P13): End-to-end system testing involves testing the entire system or application to ensure it functions as expected and meets the requirements.⁸⁸ This activity falls under continuous testing, as it helps verify the overall functionality and integration of the system.

(d) Continuous Integration (C4):

- Architecture Specifications (P1): Architecture specifications are design documents that outline the structure, components, and interactions of a software system.⁸⁹ They play a crucial role in continuous integration as they provide a blueprint for integrating code changes and maintaining system integrity.
- Automated Integration of Code Changes (P9): Automated integration of code changes refers to the process of automatically merging and integrating code changes into the main codebase.⁹⁰ It is a key practice in continuous integration, enabling frequent and reliable integration of new features or fixes.
- Code Refactoring (P16): Code refactoring involves restructuring and improving the code without changing its external behavior.⁹¹ It is a continuous process that ensures the codebase remains clean, maintainable, and adaptable. Code refactoring supports continuous integration by keeping the codebase in a healthy state.

(e) Continuous Deployment (C5):

- Automated Deployment Pipeline (P8): An automated deployment pipeline automates the process of deploying software changes to production environments.⁸⁸ It is a critical component of continuous deployment, allowing for rapid and reliable deployment of new features or bug fixes.
- Defining the Deployed Features (P11): Defining the deployed features involves specifying which features or changes will be included in a particular deployment.⁹² It is part of continuous deployment as it helps determine the scope of each deployment and ensures a smooth release process.
- Planning for Early Release (P17): Planning for early release involves considering factors like user feedback, market demands, and business goals to determine when to release new features or updates.⁹³ This activity supports continuous deployment by enabling timely and iterative releases.

(f) Continuous Monitoring (C6):

- Burn-Down Charts (P4): Burn-down charts are visual representations of the remaining work or backlog over time.⁹⁴ They are commonly used in agile development to track progress and help the team stay on track. Burn-down charts align with continuous monitoring as they provide insights into the progress of work.
- Iteration/Sprint Reviews (P14): Iteration or sprint reviews are meetings held at the end of each iteration or sprint to demonstrate completed work to stakeholders and gather feedback.⁹⁵ They contribute to continuous monitoring as they provide a checkpoint to assess progress and gather insights for future iterations.
- Limit Work in Progress (P15): Limiting work in progress (WIP) is a practice of restricting the number of tasks or user stories actively being worked on at any given time.⁹⁶ It helps maintain focus, reduce multitasking, and improve flow. Limiting WIP supports continuous monitoring by promoting a smooth and efficient workflow.

The identified practices are considered subcriteria, each mapped explicitly to a particular phase of the DevOps lifecycle considered as criteria C1, C2, C3, C4, C5, and C6. Table 8 below describes the criteria and subcriteria for framing a comparison system for choosing best practices for implementing DevOps. In summary, the categorization is based on grouping related practices and activities together to provide a structured overview of different aspects of the software development process, such as planning, development, testing, integration, deployment, and monitoring.

2. **Identifying the best and worst criteria:** According to the experts who filled empirical validation questionnaire, the best criterion determined was Continuous Integration (C1), and the worst criterion was Continuous Monitoring (C6).

In addition, experts chose Daily Stand-up (P10) and Backlog Management (P3) as the best and worst subcriteria, respectively, out of four practices grouped under Continuous Planning (C1); Collective Code Ownership (P7) and Code Review (P5) were suggested as the best and worst subcriteria, respectively, from practices grouped as Continuous Development (C2); Automated Unit Testing (P2) and End to End System Testing (P13) were suggested as the best and worst subcriteria, respectively, from a group of activities included under Continuous Testing (C3);

TABLE 8 Mapping of identified DevOps practices as criteria and subcriteria.

Criteria (C)	Subcriteria
Continuous Planning (C1)	Backlog Management (P3) Daily Standup (P10) Effective Sprint Retrospectives (P18) User Stories-based Implementation (P19)
Continuous Development (C2)	Code Review (P5) Coding Standards (P6) Collective Code Ownership (P7)
Continuous Testing (C3)	Automated Unit Testing (P2) Estimating Team Effort (P12) End-to-End System Testing (P13)
Continuous Integration (C4)	Architecture Specifications (P1) Automated Integration of Code changes (P9) Code Refactoring (P16)
Continuous Deployment (C5)	Automated Deployment Pipeline (P8) Defining the Deployed Features (P11) Planning for Early Release (P17)
Continuous Monitoring (C6)	Burn Down Charts (P4) Iteration/Sprint Reviews (P14) Limit Work in Progress (P15)

TABLE 9 Fuzzy values of preferences for best-to-others criteria and subcriteria.

Criteria	Best criteria	Other criteria	Best-to-others preferences
Main criteria (C: DevOps)	C4	C1	(1.25, 1.5, 1.75)
		C2	(1.75, 2, 2.25)
		C3	(1.25, 1.5, 1.75)
		C4	(1, 1, 1)
		C5	(0.75, 1, 1.25)
		C6	(2.25, 2.5, 2.75)
Subcriteria (C1: Continuous Planning)	P10	P3	(3.75, 4, 4.25)
		P10	(1, 1, 1)
		P18	(2.25, 2.5, 2.75)
		P19	(2.75, 3, 3.25)
Subcriteria (C2: Continuous Development)	P7	P5	(2.25, 2.5, 2.75)
		P6	(1.25, 1.5, 1.75)
		P7	(1, 1, 1)
Subcriteria (C3: Continuous Testing)	P2	P2	(1, 1, 1)
		P12	(1.75, 2, 2.25)
		P13	(2.75, 3, 3.25)
Subcriteria (C4: Continuous Integration)	P9	P1	(1.75, 2, 2.25)
		P9	(1, 1, 1)
		P16	(2.75, 3, 3.25)
Subcriteria (C5: Continuous Deployment)	P8	P8	(1, 1, 1)
		P11	(1.75, 2, 2.25)
		P17	(1.25, 1.5, 1.75)
Subcriteria (C6: Continuous Monitoring)	P14	P4	(2.75, 3, 3.25)
		P14	(1, 1, 1)
		P15	(3.25, 3.5, 3.75)

Automated Integration of Code Changes (P9) and Code Refactoring (P16) were indicated as the best and worst subcriteria, respectively, from a collection of activities included under Continuous Integration(C4); Automated Deployment Pipeline (P8) and Defining Deployed Features (P11) were chosen as the best and worst subcriteria, respectively, from a range of activities mentioned under Continuous Deployment (C5); Iteration/ Sprint Reviews (P14) and Limit Work in Progress (P15) were selected as the best and worst subcriteria, respectively, out of a range of activities mentioned under Continuous Monitoring (C6).

3. **Obtaining the preferred ranking of the best criterion over all other criteria:** The experts gave their preference in the form of linguistic terms choices presented in Table 5 to compare their chosen best criterion against each of the other criteria. Pairwise comparisons (i.e., BO preferences) for the elements of the main criterion (C), as well as six subcriteria (C1, C2, C3, C4, C5, and C6), were obtained by computing the arithmetic mean of experts. The expert ranking is displayed in Table 9.
4. **Obtaining preferred ranking of all other criteria over the worst criteria:** The phase is similar to the previous phase, but in this phase, the experts are asked to state their preferences for all other criteria with respect to the worst criterion. OW vectors represent the preferences of other criteria over the worst criteria. The aggregated OW vector was also generated by calculating the arithmetic mean of experts' surveys. Table 10 shows OW preferences for the comparison of elements of the main criteria and for all six (C1, C2, C3, C4, C5, and C6) subcriteria.

TABLE 10 Fuzzy values of preferences for others-to-worst criteria and subcriteria.

Criteria	Other criteria	Worst criteria	Other-to-Worst preferences
Main criteria (C: DevOps)	C1	C6	(0.75, 1, 1.25)
	C2		(0.75, 1, 1.25)
	C3		(0.75, 1, 1.25)
	C4		(2.25, 2.5, 2.75)
	C5		(1.25, 1.5, 1.75)
	C6		(1, 1, 1)
Subcriteria (C1: Continuous Planning)	P3	P3	(1, 1, 1)
	P10		(3.75, 4, 4.25)
	P18		(2.75, 3, 3.25)
	P19		(2.75, 3, 3.25)
Subcriteria (C2: Continuous Development)	P5	P5	(1, 1, 1)
	P6		(1.25, 1.5, 1.75)
	P7		(2.25, 2.5, 2.75)
Subcriteria (C3: Continuous Testing)	P2	P13	(2.75, 3, 3.25)
	P12		(1.75, 2, 2.25)
	P13		(1, 1, 1)
Subcriteria (C4: Continuous Integration)	P1	P16	(1.75, 2, 2.25)
	P9		(2.75, 3, 3.25)
	P16		(1, 1, 1)
Subcriteria (C5: Continuous Deployment)	P8	P11	(1.75, 2, 2.25)
	P11		(1, 1, 1)
	P17		(0.75, 1, 1.25)
Subcriteria (C6: Continuous Monitoring)	P4	P15	(1.25, 1.5, 1.75)
	P14		(3.25, 3.5, 3.75)
	P15		(1, 1, 1)

5. Calculating the optimal weights ($w_1^*, w_2^*, \dots, w_n^*$) of each of the criteria

We will use decision-makers' preferences for Best versus Others and Others versus Worst to calculate fuzzy criteria weights for the main criteria mentioned in Tables 9 and 10. According to Tables 9 and 10, fuzzy best-to-others and OW vectors are obtained as follows:

$$K_{B-O} = [(1.25, 1.5, 1.75), (1.75, 2, 2.25), (1.25, 1.5, 1.75), (1, 1, 1), (0.75, 1, 1.25), (2.25, 2.5, 2.75)]$$

$$K_{O-W} = [(0.75, 1, 1.25), (0.75, 1, 1.25), (0.75, 1, 1.25), (2.25, 2.5, 2.75), (1.25, 1.5, 1.75), (1, 1, 1)]$$

Then, the above values are put in the following non-linear constrained optimization problem.

$$\begin{aligned} & \min_{\xi^*} \\ & \text{s.t.} \left\{ \begin{array}{l} \left| \frac{(l_4^w, m_4^w, u_4^w)}{(l_1^w, m_1^w, u_1^w)} - (l_{41}, m_{41}, u_{41}) \right| \leq (k^*, k^*, k^*) \\ \left| \frac{(l_4^w, m_4^w, u_4^w)}{(l_2^w, m_2^w, u_2^w)} - (l_{42}, m_{42}, u_{42}) \right| \leq (k^*, k^*, k^*) \\ \left| \frac{(l_4^w, m_4^w, u_4^w)}{(l_3^w, m_3^w, u_3^w)} - (l_{43}, m_{43}, u_{43}) \right| \leq (k^*, k^*, k^*) \\ \left| \frac{(l_4^w, m_4^w, u_4^w)}{(l_4^w, m_4^w, u_4^w)} - (l_{44}, m_{44}, u_{44}) \right| \leq (k^*, k^*, k^*) \\ \left| \frac{(l_4^w, m_4^w, u_4^w)}{(l_5^w, m_5^w, u_5^w)} - (l_{45}, m_{45}, u_{45}) \right| \leq (k^*, k^*, k^*) \\ \left| \frac{(l_4^w, m_4^w, u_4^w)}{(l_6^w, m_6^w, u_6^w)} - (l_{46}, m_{46}, u_{46}) \right| \leq (k^*, k^*, k^*) \\ \left| \frac{(l_1^w, m_1^w, u_1^w)}{(l_6^w, m_6^w, u_6^w)} - (l_{16}, m_{16}, u_{16}) \right| \leq (k^*, k^*, k^*) \\ \left| \frac{(l_2^w, m_2^w, u_2^w)}{(l_6^w, m_6^w, u_6^w)} - (l_{26}, m_{26}, u_{26}) \right| \leq (k^*, k^*, k^*) \\ \left| \frac{(l_3^w, m_3^w, u_3^w)}{(l_6^w, m_6^w, u_6^w)} - (l_{36}, m_{36}, u_{36}) \right| \leq (k^*, k^*, k^*) \\ \left| \frac{(l_4^w, m_4^w, u_4^w)}{(l_6^w, m_6^w, u_6^w)} - (l_{46}, m_{46}, u_{46}) \right| \leq (k^*, k^*, k^*) \\ \left| \frac{(l_5^w, m_5^w, u_5^w)}{(l_6^w, m_6^w, u_6^w)} - (l_{56}, m_{56}, u_{56}) \right| \leq (k^*, k^*, k^*) \\ \left| \frac{(l_6^w, m_6^w, u_6^w)}{(l_6^w, m_6^w, u_6^w)} - (l_{66}, m_{66}, u_{66}) \right| \leq (k^*, k^*, k^*) \\ \sum_{j=1}^n R(w_j) = 1 \\ l_j^w \leq m_j^w \leq u_j^w \\ l_j^w \geq 0 \\ j = 1, 2, \dots, n \end{array} \right. \end{aligned}$$

After putting the concrete values from the B-O vector and O-W vector, the above-constrained problem will decompose into the following form, which is easily solved using Lingo software.

mink*

$$\begin{aligned}
& l_4 - 1.25 * u_1 \leq k * u_1; l_4 - 1.25 * u_1 \geq -k * u_1; \\
& m_4 - 1.5 * m_{threats1} \leq k * m_1; m_4 - 1.25 * m_1 \geq -k * m_1; \\
& u_4 - 1.75 * l_1 \leq k * l_1; u_4 - 1.25 * l_1 \geq -k * l_1; \\
& l_4 - 1.75 * u_2 \leq k * u_2; l_4 - 1.75 * u_2 \geq -k * u_2; \\
& m_4 - 2 * m_2 \leq k * m_2; m_4 - 2 * m_2 \geq -k * m_2; \\
& u_4 - 2.25 * l_2 \leq k * l_2; u_4 - 2.25 * l_2 \geq -k * l_2; \\
& l_4 - 1.25 * u_3 \leq k * u_3; l_4 - 1.25 * u_3 \geq -k * u_3; \\
& m_4 - 1.5 * m_3 \leq k * m_3; m_4 - 1.5 * m_3 \geq -k * m_3; \\
& u_4 - 1.75 * l_3 \leq k * l_3; u_4 - 1.75 * l_3 \geq -k * l_3; \\
& l_4 - 0.75 * u_5 \leq k * u_5; l_4 - 0.75 * u_5 \geq -k * u_5; \\
& m_4 - 1 * m_5 \leq k * m_5; m_4 - 1.5 * m_5 \geq -k * m_5; \\
& u_4 - 1.25 * l_5 \leq k * l_5; u_4 - 1.75 * l_5 \geq -k * l_5; \\
& l_4 - 2.25 * u_6 \leq k * u_6; l_4 - 2.25 * u_6 \geq -k * u_6; \\
& m_4 - 2.5 * m_6 \leq k * m_6; m_4 - 2.5 * m_6 \geq -k * m_6; \\
& u_4 - 2.75 * l_6 \leq k * l_6; u_4 - 2.75 * l_6 \geq -k * l_6; \\
& l_1 - 0.75 * u_6 \leq k * u_6; l_1 - 0.75 * u_6 \geq -k * u_6; \\
& m_1 - 1 * m_6 \leq k * m_6; m_1 - 1 * m_6 \geq -k * m_6; \\
& u_1 - 1.25 * l_6 \leq k * l_6; u_1 - 1.25 * l_6 \geq -k * l_6; \\
& l_2 - 0.75 * u_6 \leq k * u_6; l_2 - 0.75 * u_6 \geq -k * u_6; \\
& m_2 - 1 * m_6 \leq k * m_6; m_2 - 1 * m_6 \geq -k * m_6; \\
& u_2 - 1.25 * l_6 \leq k * l_6; u_2 - 1.25 * l_6 \geq -k * l_6; \\
& l_3 - 0.75 * u_6 \leq k * u_6; l_3 - 0.75 * u_6 \geq -k * u_6; \\
& m_3 - 1 * m_6 \leq k * m_6; m_3 - 1 * m_6 \geq -k * m_6; \\
& u_3 - 1.25 * l_6 \leq k * l_6; u_3 - 1.25 * l_6 \geq -k * l_6; \\
& l_5 - 1.25 * u_6 \leq k * u_6; l_5 - 1.25 * u_6 \geq -k * u_6; \\
& m_5 - 1.5 * m_6 \leq k * m_6; m_5 - 1.5 * m_6 \geq -k * m_6; \\
& u_5 - 1.75 * l_6 \leq k * l_6; u_5 - 1.75 * l_6 \geq -k * l_6; \\
& \frac{1}{6} * l_1 + \frac{1}{6} * 4 * m_1 + \frac{1}{6} * u_1 + \frac{1}{6} * l_2 + \frac{1}{6} * 4 * m_2 + \frac{1}{6} * u_2 + \frac{1}{6} * l_3 + \frac{1}{6} * 4 * m_3 + \frac{1}{6} * u_3 + \\
& \frac{1}{6} * l_4 + \frac{1}{6} * 4 * m_4 + \frac{1}{6} * u_4 + \frac{1}{6} * l_5 + \frac{1}{6} * 4 * m_5 + \frac{1}{6} * u_5 + \frac{1}{6} * l_6 + \frac{1}{6} * 4 * m_6 + \frac{1}{6} * u_6 = 1; \\
& l_1 \leq m_1 \leq u_1; \\
& l_2 \leq m_2 \leq u_2; \\
& l_3 \leq m_3 \leq u_3; \\
& l_4 \leq m_4 \leq u_4; \\
& l_5 \leq m_5 \leq u_5; \\
& l_6 \leq m_6 \leq u_6; \\
& l_1 \geq 0; l_2 \geq 0; l_3 \geq 0; l_4 \geq 0; l_5 \geq 0; l_6 \geq 0; \\
& k \geq 0;
\end{aligned}$$

The fuzzy optimal weights of six clusters of main criteria C1, C2, C3, C4, C5, and C6 were obtained by solving the above equation. Then, we converted these fuzzy weights into crisp weights by applying GMIR (graded mean integration representation).⁵⁴

$$GMIR(l, m, u) = \frac{l + 4m + u}{6}$$

Also, we determined the consistency ratio for each cluster with the help of their corresponding k values.

The fuzzy weights determined for the main criteria alternatives (C1, C2, C3, C4, C5, and C6) are shown in Table 11.

Similarly, we have calculated the fuzzy weights of all subcriteria alternatives (i.e., software practices for DevOps) that are grouped into six criteria. The calculated fuzzy weights of practices placed under groups of subcriteria C1–C6 are shown in Table 12.

TABLE 11 Priority weights of main criteria alternatives.

Main criteria (C)	Fuzzy weights			Crisp weight	Rank
Continuous Planning (C1)	0.1408751	0.1439369	0.1659422	0.14709415	3
Continuous Development (C2)	0.1133398	0.1133398	0.1256274	0.115387733	6
Continuous Testing (C3)	0.1408751	0.1439369	0.1659422	0.14709415	4
Continuous Integration (C4)	0.2585513	0.2602496	0.2899324	0.264913683	1
Continuous Deployment (C5)	0.186083	0.1989553	0.2443587	0.20437715	2
Continuous Monitoring (C6)	0.1187314	0.1187314	0.1331421	0.121133183	5

CR = 0.05823

TABLE 12 Priority weights of subcriteria alternatives.

Subcriteria	Practices	Fuzzy weights			Crisp weight	CR
(C1: Continuous Planning)	P3	0.1002418	0.1002418	0.1058392	0.101174700	0.07532
	P10	0.4609861	0.4609861	0.4867275	0.465276333	
	P18	0.2269688	0.2400258	0.2655875	0.242109917	
	P19	0.1740492	0.1899049	0.2149653	0.191439017	
(C2: Continuous Development)	P5	0.1852307	0.2020354	0.2054083	0.199796767	0.01391
	P6	0.271867	0.3164265	0.3377764	0.312558233	
	P7	0.447062	0.4957616	0.4957616	0.487645000	
(C3: Continuous Testing)	P2	0.4934537	0.5424664	0.5424664	0.534297617	0.03119
	P12	0.2605708	0.302836	0.3201567	0.298678583	
	P13	0.1568406	0.1690605	0.1690605	0.167023850	
(C4: Continuous Integration)	P1	0.2605708	0.302836	0.3201567	0.298678583	0.03119
	P9	0.4934537	0.5424664	0.5424664	0.534297617	
	P16	0.1568406	0.1690605	0.1690605	0.167023850	
(C5: Continuous Deployment)	P8	0.4611202	0.4611202	0.4934919	0.466515483	0.028467
	P11	0.2350634	0.2427988	0.2883083	0.249094483	
	P17	0.2596505	0.2793648	0.3292301	0.284389967	
(C6: Continuous Monitoring)	P4	0.187658	0.216662	0.2419951	0.216050183	0.02997
	P14	0.6169688	0.6169688	0.6169688	0.616968800	
	P15	0.1561753	0.1667262	0.178806	0.166981017	

The computed weights for each DevOps practice (as subcriteria alternatives) listed above were multiplied by the weights of the corresponding main criteria alternatives to obtain the final weights and the final ranking of DevOps practices. The results section discusses the final DevOps practices rankings.

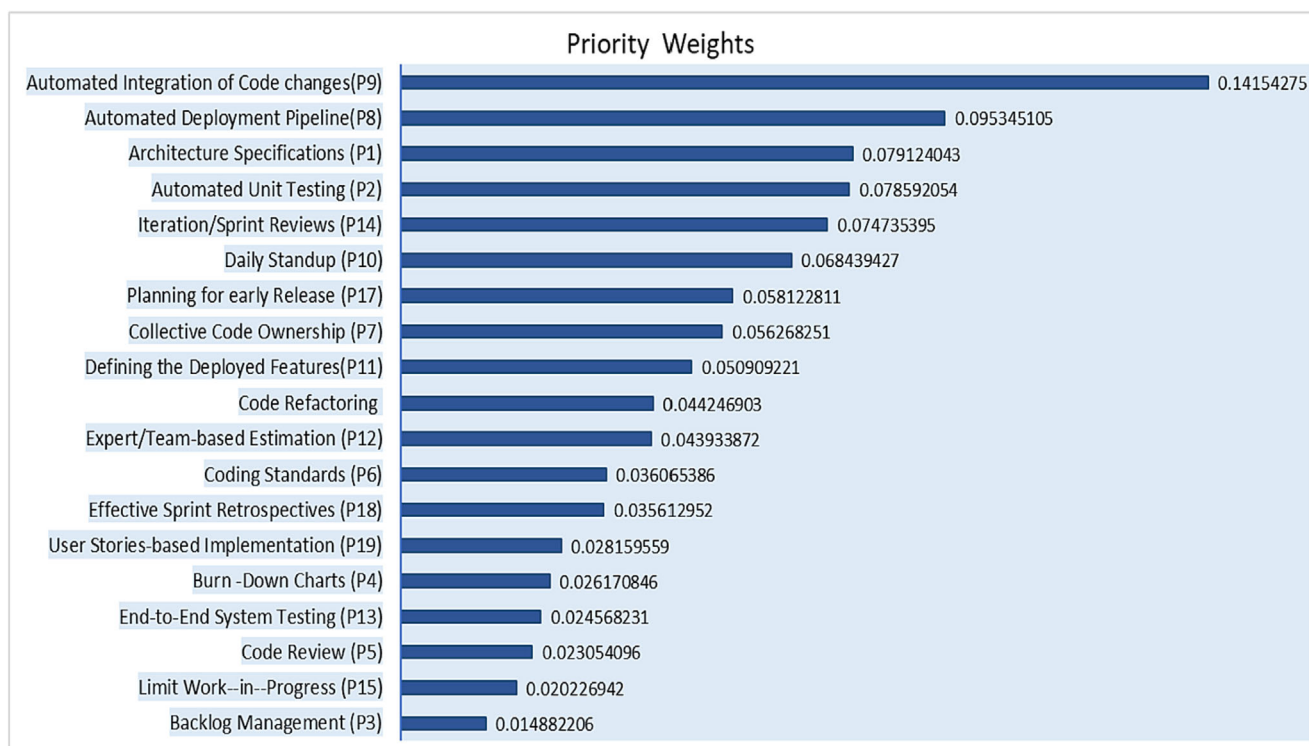
This section analyses the criteria and subcriteria, and then the results of ranking software development practices for DevOps implementation are discussed. To determine the final weights and ranking of the DevOps implementation practices, the weight obtained in the previous step for each practice was multiplied by the weight of the corresponding primary criterion (i.e., DevOps phases weight). The final ranking of the practices was then determined using the obtained weights. The outcomes of these calculations and the final rankings of the DevOps implementation practices are shown in Table 13 and Figure 7. The consistency ratio was calculated by dividing the K values obtained from the best-to-worst criteria ratio by the corresponding values at each solution stage. At each step, this ratio was less than 0.1.

5 | DISCUSSION AND SUMMARY

The study reported software practices for DevOps implementation within a software organization. The study's results are summarized as follows:

TABLE 13 Final ranking of DevOps practices using FBWM.

S. No.	Categories	Weights	DevOps implementation practices	Local weights	Global weights	Rank
1.	Continuous Planning (C1)	0.14709415	Backlog Management (P3)	0.1011747	0.014882206	19
			Daily Standup (P10)	0.465276333	0.068439427	6
			Effective Sprint Retrospectives (P18)	0.242109917	0.035612952	13
			User Stories Based Implementation(P19)	0.191439017	0.028159559	14
2.	Continuous Development (C2)	0.115387733	Code Review (P5)	0.199796767	0.023054096	17
			Coding Standards (P6)	0.312558233	0.036065386	12
			Collective Code Ownership (P7)	0.487645	0.056268251	8
3.	Continuous Testing (C3)	0.14709415	Automated Unit Testing (P2)	0.534297617	0.078592054	4
			Expert/Team-based Estimation (P12)	0.298678583	0.043933872	11
			End-to-End System Testing (P13)	0.16702385	0.024568231	16
4.	Continuous Integration (C4)	0.264913683	Architecture Specifications (P1)	0.298678583	0.079124043	3
			Automated Integration of Code Changes (P9)	0.534297617	0.14154275	1
			Code Refactoring (P16)	0.16702385	0.044246903	10
5.	Continuous Deployment (C5)	0.20437715	Automated Deployment Pipeline (P8)	0.466515483	0.095345105	2
			Defining Deployed Features (P11)	0.249094483	0.050909221	9
			Planning for Early Release (P17)	0.284389967	0.058122811	7
6.	Continuous Monitoring (C6)	0.121133183	Burn Down Charts (P4)	0.216050183	0.026170846	15
			Iteration/Sprint Reviews (P14)	0.6169688	0.074735395	5
			Limit Work in Progress (P15)	0.166981017	0.020226942	18

**FIGURE 7** Final weights of DevOps practices.

5.1 | RQ1 (identified software practices)

We conducted an SLR to explore the existing DevOps literature to determine the DevOps practices the academics reported. We have identified a total of 19 software practices that must be addressed for the successful implementation and deployment of DevOps. The identified practices for DevOps implementation are shown in Table 7.

5.1.1 | RQ1.1 (categorization of identified software practice)

The software practices determined from the Systematic literature review were used as decision criteria to apply the Best-worst method. The number of software practices was 19, but for criteria greater than 9, BMW's developers recommend grouping the criteria into a number of clusters. This way, one more level is added to the hierarchy of the problem. We conceptually map the identified practices into the six stages of the DevOps lifecycle to execute the clustering, as shown in Table 8. The identified software practices were mapped by considering two issues: the relationship of the identified software practice to the DevOps phases and the mapping team's level of understanding. All the authors of this study were involved in the mapping process. The 19 practices were evaluated and classified into one of the six DevOps lifecycle categories, as shown in Table 8.

5.2 | RQ2 (significance of identified software practice)

Since the SLR has identified these software practices, it is clear that they are the software practices that most academics and researchers have discussed. Additionally, the DevOps industry experts have verified these identified practices. According to empirical validation, these approaches are frequently used to implement DevOps in the real-world software development industry. The practices identified are essential components of implementing DevOps. In summary, implementing these practices in DevOps can help teams improve collaboration, agility, and quality in software development and delivery. By adopting a holistic approach to software development, teams can deliver software faster, with fewer defects, and with a higher degree of confidence.

5.3 | RQ3 (prioritization of identified software practice)

To rank the identified practices and their categories in order of priority, we followed the step-by-step protocols of the FBWM. We conducted a pairwise comparison with the experts to determine the ranks of the identified practices and their related categories. The experts considered the FBWM-calculated weights (i.e., priorities) when rating the identified practices and the categories they fall under. The FBWM approach offers a thorough understanding of decision-making problems that consider DevOps practices and the categories that go along with them. The highest ranked software practices for DevOps implementation, as shown in Table 13, are “Automated Integration of Code Changes” with a final weight of 0.14154275 and “Automated Deployment Pipeline” with a final weight of 0.095345105. In other words, these two practices are significant for the growth and advancement of DevOps in the software organization. Architecture Specifications and Automated Unit Testing, with final weights of 0.079124043 and 0.078592054, are the other two most crucial practices in this regard. In contrast, among the considered DevOps practices, practices like “Backlog Management” and “Limit work in Progress,” with final weights of 0.014882206 and 0.020226942, respectively, were identified as the least important for the implementation of DevOps.

Additionally, Table 14 illustrates the prioritization of both DevOps lifecycle phases and corresponding DevOps practices. The table's first column presents the DevOps lifecycle phases, arranged in ascending order based on their priority. Within each phase, the associated practices are also listed in order of their relative importance. In essence, Table 14 provides a comprehensive overview of how different DevOps stages are ranked in terms of their significance, with a further breakdown of the prioritized practices within each phase. This valuable information can assist teams in understanding the critical focus areas and the most essential practices to implement for successful DevOps implementation and deployment.

5.4 | RQ4 (prioritization taxonomy of identified software practice)

The taxonomy of software practices for applying DevOps in software organizations was developed based on the findings of the BWM analysis, as shown in Figure 8. The framework developed for this study was based on 19 software practices, divided into six groups: Continuous Planning, Continuous Development, Continuous Testing, Continuous Integration, Continuous Deployment, and Continuous Monitoring. As per Table 11,

TABLE 14 Priorities of identified development practices.

DevOps lifecycle phases priority	DevOps practices		
	Higher priority	Moderate priority	Lower priority
Continuous Integration (C4)	Automated Integration of Code Changes (P9)	Architecture Specifications (P1)	Code Refactoring (P16)
Continuous Deployment (C5)	Automated Deployment Pipeline (P8)	Planning for Early Release (P17)	Defining Deployed Features (P11)
Continuous Planning (C1)	Daily Standup (P10)	Effective Sprint Retrospectives (P18)	User Stories Based Implementation (P19), Backlog Management (P3)
Continuous Testing (C3)	Automated Unit Testing (P2)	Expert/Team-based Estimation (P12)	End-to-End System Testing (P13)
Continuous Monitoring (C6)	Limit Work in Progress (P15)	Burn Down Charts (P4)	Limit Work in Progress (P15)
Continuous Development (C2)	Collective Code Ownership (P7)	Coding Standards (P6)	Code Review (P5)

the categories of Continuous Integration, Continuous Deployment, Continuous Testing, Continuous Planning, Continuous Monitoring, and Continuous Development are ranked from first to sixth and are weighted by 0.264913683, 0.20437715, 0.121133183, 0.14709415, 0.14709415, and 0.115387733, respectively. The developed taxonomy of practices is shown in Figure 8, which indicates that the “Continuous Integration,” with a weight of 0.264913683, is in the first rank within the main criterion, according to the calculated weight (Table 11). This indicates that Continuous Integration is the most crucial area that practitioners should focus on to implement and manage DevOps in software organizations successfully. Also, “Continuous Development” was determined to be the least significant category, with a weighting of 0.1153877, indicating that phases of the Operations side are more significant than the Development side.

Moreover, the taxonomy (Figure 8) provides the rankings for each identified practice in the local and global contexts. This shows the importance of each practice within the category (Table 13). As a subcriterion, P9 (Automated Integration of Code Changes) is ranked as the highest software practice within the Continuous Integration category (C4), which is also represented as a criterion as it relates to the DevOps lifecycle (local rank) as well as among 19 practices (global rank). Automated Deployment Pipeline (P8), which was selected as the best software practice out of a range of practices specified under the Continuous Deployment (C5) criterion (local rank), was ranked second out of 19 practices (global rank). Automated Unit Testing (P2) was ranked first (local rank) in the group of practices included under Continuous Testing (C3) and ranked at four (global rank) within 19 practices. Out of the four practices included in Continuous Planning (C1), Daily Stand-up (P10) had the highest ranking (local rank) and finished sixth overall among all practices (global rank). Out of a variety of practices listed under the Continuous Monitoring (C6) category, Iteration/Sprint Reviews (P14) is ranked first within the category (local rank) and sixth among all 19 practices (global rank). Collective Code Ownership (P7) is rated first among practices grouped as Continuous Development (C2) and ranked 12th among all 19 practices.

6 | THREATS TO VALIDITY AND LIMITATIONS

In this section, we will focus on addressing threats to the validity of our study. Despite careful methodology, there are inherent risks to validity that need to be considered.

6.1 | Internal threat

The one internal validity of our study is the researcher's bias and inconsistent data extraction techniques, which could undermine the study's conclusions. To address this concern, we have involved multiple authors in the data extraction process to minimize the potential impact of these threats. Another internal validity threat stems from the limited availability of data from diverse sources, which increases the risk of missing crucial information. Although increasing the percentage of newer references is desirable, ensuring the quality and relevance of the included publications is equally important. Merely including recent articles without assessing their contribution and rigor may lead to a biased or skewed review. Quality criteria, such as peer-reviewed journals and reputable conferences, have been applied to maintain the rigor of the SLR. To mitigate this threat, we employed a search string to collect information from multiple sources, reducing the potential impact of this limitation on internal validity. Although our study is on DevOps practices, it did not explicitly address the latest practices in the field. This limitation could be due to DevOps, as a field continuously evolving, and new techniques and practices are constantly emerging to improve software development and delivery processes.

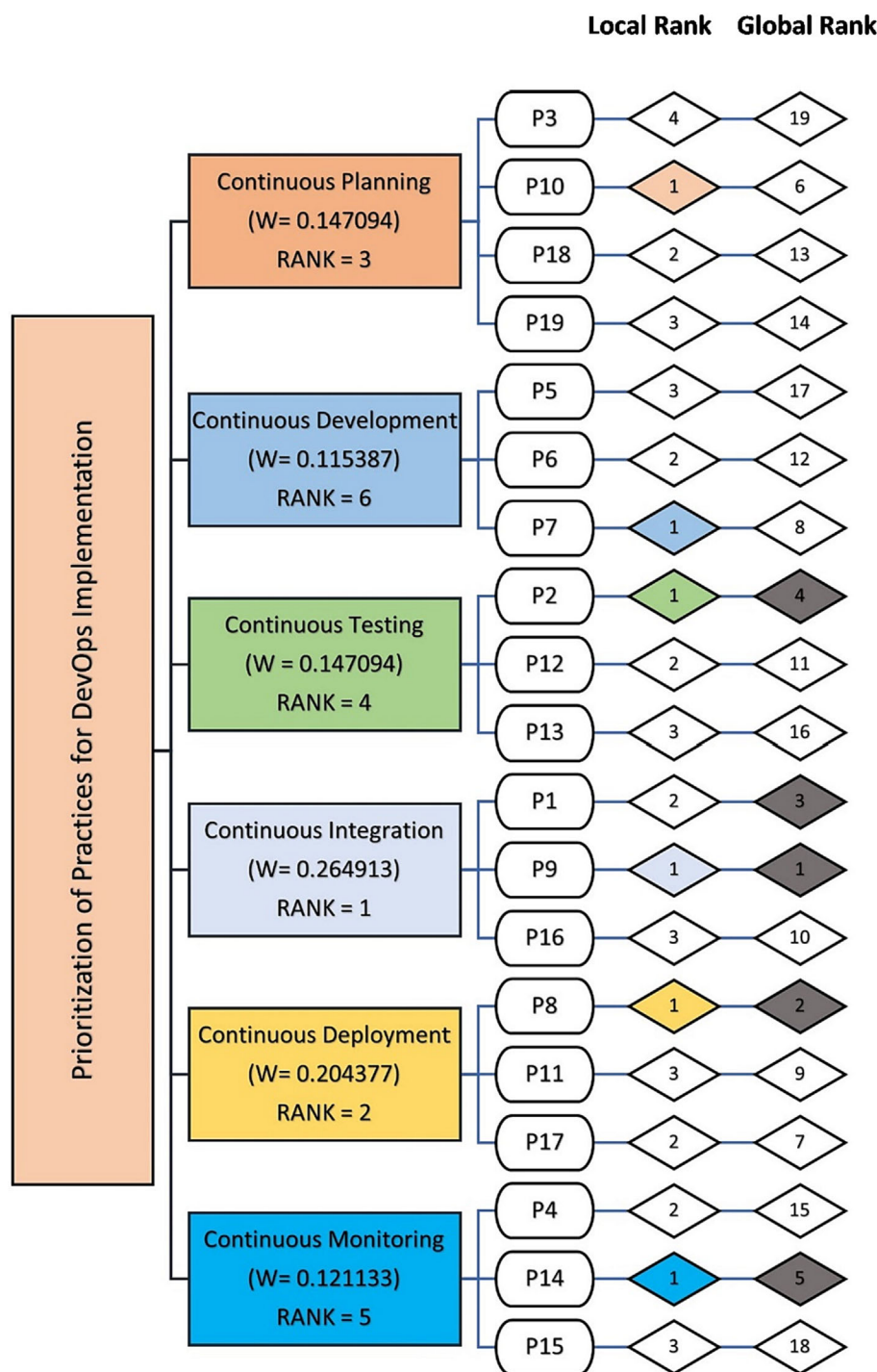


FIGURE 8 Taxonomy of prioritization of software practices for DevOps implementation.

6.2 | External threat

Moreover, the external validity of the study is a concern due to potential limitations in generalizing the results. Although the majority of survey respondents are from Asia, there is a smaller sample of participants from non-Asian organizations. As such, the generalizability of the study's findings is somewhat limited. It is worth noting that the DevOps practices used in the study were derived from the latest accessible research and were validated through an empirical investigation involving professionals. The responses from the survey participants further supported the relevance of the reported software practices to their work. However, it is important to acknowledge that the majority of survey study participants come from developing nations.

7 | IMPLICATIONS OF THE STUDY

This section includes the implications of the study for both DevOps practitioners and researchers. Researchers and practitioners can use the knowledge gained from this study to create successful plans and strategies for successful DevOps deployment. From the practitioner's perspective, the results of the study are explored by using SLR and empirical study that provides a knowledge base related to the DevOps practices involved in the software development activities. There are 19 practices identified from the literature and validated using the questionnaire study. To evaluate the most crucial practices for implementing DevOps, FBWM was utilized to create the taxonomy of the practices and their categories. The developed taxonomy of the practices informs the practitioners about the relative significance of every identified practice. The novel application of FBWM in this domain could help industry practitioners address the MCDM problems with the vague opinions of DevOps experts. Regarding the research implication, this study provides a knowledge base that enables the research community to explore the most critical practices in developing a robust system. The identified list of practices reveals new research directions, such as the need for effective guidelines to implement these practices. The study also examines the identified practices against the core categories of the SPI manifesto, aiding researchers in developing tools and strategies to address DevOps practices at the category level.

8 | CONCLUSION AND FUTURE RESEARCH

The purpose of this article was to present a definite framework for identifying key software practices to implement DevOps in software development organizations successfully. Considering the complexity of achieving success for DevOps in the software industry, key software practices have been prioritized to emphasize the more crucial ones. Using an SLR, 19 software development practices were identified that led to the success of implementing DevOps in software organizations. According to their characteristics, the software practices were categorized into six categories: Continuous Planning, Continuous Development, Continuous Testing, Continuous Integration, Continuous Deployment, and Continuous Monitoring. A questionnaire was prepared and disseminated to experts to collect the data needed to implement FBWM to prioritize these DevOps practices. The experts were asked in this questionnaire to rank the relevance of several other practices related to these identified software practices to DevOps adoption, which they considered most and least important. After data collection using questionnaires, optimization models were built for the main criteria and subcriteria. Lingo software was used to solve them to obtain ambiguous weights for each factor. Finally, the ranking was performed after converting the calculated fuzzy weights to crisp weights. Among the considered subcriteria, "Automated Integration of Code Changes," with a final weight of 0.14154275, was identified as the most significant practice, and "Backlog management" was recognized as the least essential practice, with a final weight of 0.014882206. Our study also opens up some future directions, which are as follows:

- Examine the current state of the software practices that companies and organizations of DevOps have prioritized to identify gaps.
- Conduct a similar study by taking other DevOps-related criteria such as challenging factors, metrics, etc.
- Future studies can further the research by integrating the FBWM approach with new MCDM techniques.
- Evaluate the findings of the current study in different software industry sectors.

DATA AVAILABILITY STATEMENT

The data that support the findings of this study are available from the corresponding author upon reasonable request.

ORCID

Ankur Kumar  <https://orcid.org/0000-0002-2417-9581>

Mohammad Nadeem  <https://orcid.org/0000-0003-3664-5014>

Mohammad Shameem  <https://orcid.org/0000-0002-6055-5345>

REFERENCES

1. Lin B, Cassee N, Serebrenik A, Bavota G, Novielli N, Lanza M. Opinion mining for software development: a systematic literature review. *ACM Trans Softw Eng Methodol*. 2022;31(3):1-41. doi:[10.1145/3490388](https://doi.org/10.1145/3490388)
2. Kwan A, Jacobsen H-A, Chan A, Samooj S. Microservices in the modern software world. In: *Proceedings of the 26th Annual International Conference on Computer Science and Software Engineering*, in CASCON'16. IBM Corp.; 2016:297-299.
3. Gall M, Pigni F. Taking DevOps mainstream: a critical review and conceptual framework. *Eur J Inf Syst*. 2022;31(5):548-567. doi:[10.1080/0960085X.2021.1997100](https://doi.org/10.1080/0960085X.2021.1997100)
4. Gemino A, Horner Reich B, Serrador PM. Agile, traditional, and hybrid approaches to project success: is hybrid a poor second choice? *Proj Manag J*. 2020;52(2):161-175. doi:[10.1177/8756972820973082](https://doi.org/10.1177/8756972820973082)
5. Hemon A, Lyonnet B, Rowe F, Fitzgerald B. From agile to DevOps: smart skills and collaborations. *Information Systems Frontiers*. 2020;22(4):927-945. doi:[10.1007/s10796-019-09905-1](https://doi.org/10.1007/s10796-019-09905-1)

6. Düllmann TF, Kabierschke O, van Hoorn A. StalkCD: a model-driven framework for interoperability and analysis of CI/CD pipelines. In: *2021 47th Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*; 2021:214–223. doi:[10.1109/SEAA53835.2021.00035](https://doi.org/10.1109/SEAA53835.2021.00035)
7. “AIS electronic library (AISel)—Hawaii International Conference on System Sciences 2019 (HICSS-52): Implementing the Planning Process within DevOps Teams to Achieve Continuous Innovation.”
8. Gheorghe-Pop I-D, Tcholtchev N, Ritter T, Hauswirth M. Quantum DevOps: towards reliable and applicable NISQ quantum computing. In: *2020 IEEE Globecom Workshops (GC Wkshps)*; 2020:1–6. doi:[10.1109/GCWkshps50303.2020.9367411](https://doi.org/10.1109/GCWkshps50303.2020.9367411)
9. Shahin M, Ali Babar M, Zhu L. Continuous integration, delivery and deployment: a systematic review on approaches, tools, challenges and practices. *IEEE Access*. 2017;5:3909–3943. doi:[10.1109/ACCESS.2017.2685629](https://doi.org/10.1109/ACCESS.2017.2685629)
10. Hemon-Hildgen A, Rowe F, Monnier-Senicourt L. Orchestrating automation and sharing in DevOps teams: a revelatory case of job satisfaction factors, risk and work conditions. *Eur J Inf Syst*. 2020;29(5):474–499. doi:[10.1080/0960085X.2020.1782276](https://doi.org/10.1080/0960085X.2020.1782276)
11. Amershi S, Begel A, Bird C, et al. Software engineering for machine learning: a case study. In: *2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*; 2019:291–300. doi:[10.1109/ICSE-SEIP.2019.00042](https://doi.org/10.1109/ICSE-SEIP.2019.00042)
12. Peterson M, Summers JD. Recommended methods supporting adoption of the agile philosophy for hardware development. In: *33rd International Conference on Design Theory and Methodology (DTM)*. Vol.6. American Society of Mechanical Engineers; 2021. doi:[10.1115/DETC2021-70621](https://doi.org/10.1115/DETC2021-70621)
13. Cepuc A, Botez R, Craciun O, Ivanciu I-A, Dobrota V. Implementation of a continuous integration and deployment pipeline for containerized applications in Amazon web services using Jenkins, Ansible and Kubernetes. In: *2020 19th RoEduNet Conference: Networking in Education and Research (RoEduNet)*; 2020:1–6. doi:[10.1109/RoEduNet51892.2020.9324857](https://doi.org/10.1109/RoEduNet51892.2020.9324857)
14. Akbar MA, Smolander K, Mahmood S, Alsanad A. Toward successful DevSecOps in software development organizations: a decision-making framework. *Inf Softw Technol*. 2022;147:106894. doi:[10.1016/j.infsof.2022.106894](https://doi.org/10.1016/j.infsof.2022.106894)
15. Alla Sridhar SK, Adari. What Is MLOps? In: *Beginning MLOps with MLFlow: deploy models in AWS SageMaker, Google Cloud, and Microsoft Azure*. Apress; 2021:79–124. doi:[10.1007/978-1-4842-6549-9_3](https://doi.org/10.1007/978-1-4842-6549-9_3)
16. B. P, Iraj C-PRL, Acharya. BizDevOps: a multivocal literature review. In: B. and M. S. and G. C. and B. I. and T. D. and A. B. O. and R. A. M. A. C. and T. E. and T. C. M, Osvaldo KYG, Murgante, eds. *Computational Science and Its Applications—ICCSA*. Springer International Publishing; 2020:698–713.
17. Khan MS, Khan AW, Khan F, Khan MA, Whangbo TK. Critical challenges to adopt DevOps culture in software organizations: a systematic review. *IEEE Access*. 2022;10:14339–14349. doi:[10.1109/ACCESS.2022.3145970](https://doi.org/10.1109/ACCESS.2022.3145970)
18. Leite L, Rocha C, Kon F, Milojevic D, Meirelles P. A survey of DevOps concepts and challenges. *ACM Comput Surv*. 2019;52(6):1–35. doi:[10.1145/3359981](https://doi.org/10.1145/3359981)
19. Stahl D, Martensson T, Bosch J. Continuous practices and devops: beyond the buzz, what does it all mean? In: *2017 43rd Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*; 2017:440–448. doi:[10.1109/SEAA.2017.8114695](https://doi.org/10.1109/SEAA.2017.8114695)
20. Lwakatere LE, Kilamo T, Karvonen T, et al. DevOps in practice: a multiple case study of five companies. *Inf Softw Technol*. 2019;114:217–230. doi:[10.1016/j.infsof.2019.06.010](https://doi.org/10.1016/j.infsof.2019.06.010)
21. Senapathi M, Buchan J, Osman H. DevOps capabilities, practices, and challenges: insights from a case study. In: *Proceedings of the 22nd International Conference on Evaluation and Assessment in Software Engineering 2018, in EASE'18*. Association for Computing Machinery; 2018:57–67. doi:[10.1145/3210459.3210465](https://doi.org/10.1145/3210459.3210465)
22. Amaro RMD, Pereira R, da Silva M. Capabilities and practices in DevOps: a multivocal literature review. *IEEE Trans Softw Eng*. 2022;49(2):1–901. doi:[10.1109/TSE.2022.3166626](https://doi.org/10.1109/TSE.2022.3166626)
23. Macarthy RW, Bass JM. An empirical taxonomy of DevOps in practice. In: *2020 46th Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*; 2020:221–228. doi:[10.1109/SEAA51224.2020.00046](https://doi.org/10.1109/SEAA51224.2020.00046)
24. Callanan M, Spillane A. DevOps: making it easy to do the right thing. *IEEE Softw*. 2016;33(3):53–59. doi:[10.1109/MS.2016.66](https://doi.org/10.1109/MS.2016.66)
25. Morales JA, Yasar H, Volkman A. Implementing DevOps practices in highly regulated environments. In: *Proceedings of the 19th International Conference on Agile Software Development: Companion, in XP'18*. Association for Computing Machinery; 2018. doi:[10.1145/3234152.3234188](https://doi.org/10.1145/3234152.3234188)
26. Kumar A, Nadeem M, Shameem M. Prioritization of DevOps maturity models using Fuzzy TOPSIS. In: *Proceedings of the 27th International Conference on Evaluation and Assessment in Software Engineering*. ACM; 2023:438–443. doi:[10.1145/3593434.3594241](https://doi.org/10.1145/3593434.3594241)
27. Akbar MA, Mahmood S, Shafiq M, Alsanad A, Alsanad AA-A, Gumaei A. Identification and prioritization of DevOps success factors using fuzzy-AHP approach. *Soft Comput*. 2020;27(4):1907–1931. doi:[10.1007/s00500-020-05150-w](https://doi.org/10.1007/s00500-020-05150-w)
28. Noorani NM, Zamani AT, Alenezi M, Shameem M, Singh P. Factor prioritization for effectively implementing DevOps in software development organizations: a SWOT-AHP approach. *Axioms*. 2022;11(10):498. doi:[10.3390/axioms11100498](https://doi.org/10.3390/axioms11100498)
29. Anandya R, Raharjo T, Suhanto A. Challenges of DevOps implementation: a case study from technology companies in Indonesia. In: *2021 International Conference on Informatics, Multimedia, Cyber and Information System (ICIMCIS)*; 2021:108–113. doi:[10.1109/ICIMCIS53775.2021.9699240](https://doi.org/10.1109/ICIMCIS53775.2021.9699240)
30. Luz WP, Pinto G, Bonifácio R. Adopting DevOps in the real world: a theory, a model, and a case study. *J Syst Softw*. 2019;157:110384. doi:[10.1016/j.jss.2019.07.083](https://doi.org/10.1016/j.jss.2019.07.083)
31. Kumar A, Nadeem M, Shameem M. Machine learning based predictive modeling to effectively implement DevOps practices in software organizations. *Automated Softw Eng*. 2023;30(2):21. doi:[10.1007/s10515-023-00388-8](https://doi.org/10.1007/s10515-023-00388-8)
32. Elo J, Pekkala K, Tuunanen T, Lumivalo J, Salo M. Exploring the “why”, “how”, and “what” of continuous digital service innovation. In: M. and M. T. and K. J, Davide APT, Kuhrmann, eds. *Product-Focused Software Process Improvement*. Springer International Publishing; 2022:366–381. doi:[10.1007/978-3-031-21388-5_25](https://doi.org/10.1007/978-3-031-21388-5_25)
33. Dörnenburg E. The path to DevOps. *IEEE Softw*. 2018;35(5):71–75. doi:[10.1109/MS.2018.290110337](https://doi.org/10.1109/MS.2018.290110337)
34. Rafi S, Akbar MA, Mahmood S, Alsanad A, Alothaim A. Selection of DevOps best test practices: a hybrid approach using ISM and fuzzy TOPSIS analysis. *J Softw Evol Process*. 2022;34(5):2448. doi:[10.1002/smr.2448](https://doi.org/10.1002/smr.2448)
35. Piccarozzi M, Aquilani B, Gatti C. Industry 4.0 in management studies: a systematic literature review. *Sustainability*. 2018;10(10):3821. doi:[10.3390/su10103821](https://doi.org/10.3390/su10103821)
36. Kitchenham B, Pearl Brereton O, Budgen D, Turner M, Bailey J, Linkman S. Systematic literature reviews in software engineering—a systematic literature review. *Inf Softw Technol*. 2009;51(1):7–15. doi:[10.1016/j.infsof.2008.09.009](https://doi.org/10.1016/j.infsof.2008.09.009)
37. Turner M, Kitchenham B, Brereton P, Charters S, Budgen D. Does the technology acceptance model predict actual use? A systematic literature review. *Inf Softw Technol*. 2010;52(5):463–479. doi:[10.1016/j.infsof.2009.11.005](https://doi.org/10.1016/j.infsof.2009.11.005)

38. Chen L, Babar MA, Zhang HN. Towards an evidence-based understanding of electronic data sources; 2010. doi:[10.14236/ewic/EASE2010.17](https://doi.org/10.14236/ewic/EASE2010.17)
39. Rahman A, Mahdavi-Hezaveh R, Williams L. A systematic mapping study of infrastructure as code research. *Inf SoftwTechnol*. 2019;108:65-77. doi:[10.1016/j.infsof.2018.12.004](https://doi.org/10.1016/j.infsof.2018.12.004)
40. Mao R, Zhang H, Dai Q, et al. Preliminary findings about DevSecOps from grey literature. In: 2020 IEEE 20th International Conference on Software Quality, Reliability And Security (QRS); 2020:450-457. doi:[10.1109/QRS51102.2020.00064](https://doi.org/10.1109/QRS51102.2020.00064)
41. Muñoz M, Negrete M, Mejía J. Proposal to avoid issues in the DevOps implementation: a systematic literature review. In: H. and R. L. P, Álvaro CSR, Adeli, eds. *New knowledge in information systems and technologies*. Springer International Publishing; 2019:666-677. doi:[10.1007/978-3-030-16181-1_63](https://doi.org/10.1007/978-3-030-16181-1_63)
42. Badshah S, Khan AA, Khan B. Towards process improvement in DevOps: a systematic literature review. In: *Proceedings of the Evaluation and Assessment in Software Engineering, in EASE'20*. Association for Computing Machinery; 2020:427-433. doi:[10.1145/3383219.3383280](https://doi.org/10.1145/3383219.3383280)
43. Kumar A, Nadeem M, Shameem M. Systematic literature review of metrics for measuring devops success; 2023:030005. doi:[10.1063/5.0128883](https://doi.org/10.1063/5.0128883)
44. Zhou Y, Zhang H, Huang X, Yang S, Babar MA, Tang H. Quality assessment of systematic reviews in software engineering: a tertiary study. In: *Proceedings of the 19th International Conference on Evaluation and Assessment in Software Engineering, in EASE'15*. Association for Computing Machinery; 2015. doi:[10.1145/2745802.2745815](https://doi.org/10.1145/2745802.2745815)
45. Bin Ali N, Usman M. Reliability of search in systematic reviews: towards a quality assessment framework for the automated-search strategy. *Inf SoftwTechnol*. 2018;99:133-147. doi:[10.1016/j.infsof.2018.02.002](https://doi.org/10.1016/j.infsof.2018.02.002)
46. Yang L, Zhang H, Shen H, et al. Quality assessment in systematic literature reviews: a software engineering perspective. *Inf SoftwTechnol*. 2021;130:106397. doi:[10.1016/j.infsof.2020.106397](https://doi.org/10.1016/j.infsof.2020.106397)
47. Afzal W, Torkar R, Feldt R. A systematic review of search-based testing for non-functional system properties. *Inf SoftwTechnol*. 2009;51(6):957-976. doi:[10.1016/j.infsof.2008.12.005](https://doi.org/10.1016/j.infsof.2008.12.005)
48. Lin aker J, Sulaman SM, de Mello RM, Höst M. *Guidelines for Conducting Surveys in Software Engineering*. Lund University; 2016.
49. Groves RM, Fowler FJ Jr, Couper MP, Lepkowski JM, Singer E, Tourangeau R. *Survey Methodology*. Google Books.
50. Akdur D, Garousi V, Demirörs O. A survey on modeling and model-driven engineering practices in the embedded software industry. *J Syst Architecture*. 2018;91:62-82. doi:[10.1016/j.sysarc.2018.09.007](https://doi.org/10.1016/j.sysarc.2018.09.007)
51. Pisotta I, Tagliamonte NL, Bigioni A, et al. Pilot testing of a new questionnaire for the assessment of user experience during exoskeleton-assisted walking; 2022:195-199. doi:[10.1007/978-3-030-70316-5_32](https://doi.org/10.1007/978-3-030-70316-5_32)
52. Harkness J, Pennell B-E, Schoua-Glusberg A. Survey questionnaire translation and assessment. In: *Methods for Testing and Evaluating Survey Questionnaires*. John Wiley & Sons, Inc.; 453-473. doi:[10.1002/0471654728.ch22](https://doi.org/10.1002/0471654728.ch22)
53. Nicholas D, Jamali HR, Herman E, et al. A global questionnaire survey of the scholarly communication attitudes and behaviours of early career researchers. *Learned Publishing*. 2020;33(3):198-211. doi:[10.1002/leap.1286](https://doi.org/10.1002/leap.1286)
54. Chiao K-P. Ranking interval type 2 fuzzy sets using parametric graded mean integration representation. In: 2016 International Conference on Machine Learning and Cybernetics (ICMLC). Vol.2016:606-611. doi:[10.1109/ICMLC.2016.7872956](https://doi.org/10.1109/ICMLC.2016.7872956)
55. Ma Z, Yan L. Data modeling and querying with fuzzy sets: a systematic survey. *Fuzzy Set Syst*. 2022;445:147-183. doi:[10.1016/j.fss.2022.01.006](https://doi.org/10.1016/j.fss.2022.01.006)
56. Abraham S, Punniyamoorthy M. A fuzzy approach using asymmetrical triangular distribution in a two-person zero-sum game for a multi-criteria decision-making problem. *Quantum Mach Intell*. 2021;3(1):20. doi:[10.1007/s42484-021-00044-y](https://doi.org/10.1007/s42484-021-00044-y)
57. Ananthi VP, Balasubramaniam P, Lim CP. Segmentation of gray scale image based on intuitionistic fuzzy sets constructed from several membership functions. *Pattern Recognit*. 2014;47(12):3870-3880. doi:[10.1016/j.patcog.2014.07.003](https://doi.org/10.1016/j.patcog.2014.07.003)
58. Rezaei J. Best-worst multi-criteria decision-making method. *Omega (Westport)*. 2015;53:49-57. doi:[10.1016/j.omega.2014.11.009](https://doi.org/10.1016/j.omega.2014.11.009)
59. Youssef AE. An integrated MCDM approach for cloud service selection based on TOPSIS and BWM. *IEEE Access*. 2020;8:71851-71865. doi:[10.1109/ACCESS.2020.2987111](https://doi.org/10.1109/ACCESS.2020.2987111)
60. Guo S, Zhao H. Fuzzy best-worst multi-criteria decision-making method and its applications. *Knowl Based Syst*. 2017;121:23-31. doi:[10.1016/j.knosys.2017.01.010](https://doi.org/10.1016/j.knosys.2017.01.010)
61. Dong J, Wan S, Chen S-M. Fuzzy best-worst method based on triangular fuzzy numbers for multi-criteria decision-making. *Inf Sci (N Y)*. 2021;547:1080-1104. doi:[10.1016/j.ins.2020.09.014](https://doi.org/10.1016/j.ins.2020.09.014)
62. Sarabi EP, Darestani SA. Developing a decision support system for logistics service provider selection employing fuzzy MULTIMOORA & BWM in mining equipment manufacturing. *Appl Soft Comput*. 2021;98:106849. doi:[10.1016/j.asoc.2020.106849](https://doi.org/10.1016/j.asoc.2020.106849)
63. Odu GO. Weighting methods for multi-criteria decision making technique. *J Appl Sci Environ Manag*. 2019;23(8):1449. doi:[10.4314/jasem.v23i8.7](https://doi.org/10.4314/jasem.v23i8.7)
64. Amiri M, Hashemi-Tabatabaei M, Keshavarz-Ghorabae M, Kaklauskas A, Zavadskas EK, Antucheviciene J. A fuzzy extension of simplified best-worst method (F-SBWM) and its applications to decision-making problems. *Symmetry (Basel)*. 2022;15(1):81. doi:[10.3390/sym15010081](https://doi.org/10.3390/sym15010081)
65. Liang F, Brunelli M, Rezaei J. Consistency issues in the best worst method: measurements and thresholds. *Omega (Westport)*. 2020;96:102175. doi:[10.1016/j.omega.2019.102175](https://doi.org/10.1016/j.omega.2019.102175)
66. Hernández R, Moros B, Nicolás J. Requirements management in DevOps environments: a multivocal mapping study. *Requir Eng*. 2023;28(3):317-346. doi:[10.1007/s00766-023-00396-w](https://doi.org/10.1007/s00766-023-00396-w)
67. Ibrahim MMA, Syed-Mohamad SM, Husin MH. Managing quality assurance challenges of DevOps through analytics. In: *Proceedings of the 2019 8th International Conference on Software and Computer Applications*. ACM; 2019:194-198. doi:[10.1145/3316615.3316670](https://doi.org/10.1145/3316615.3316670)
68. Rafi S, Akbar MA, Sánchez-Gordón M, Colomo-Palacios R. DevOps practitioners' perceptions of the low-code trend. In: *ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*. ACM; 2022:301-306. doi:[10.1145/3544902.3546635](https://doi.org/10.1145/3544902.3546635)
69. Shahin M. Architecting for DevOps and continuous deployment. In: *Proceedings of the ASWEC 2015 24th Australasian Software Engineering Conference, in ASWEC' 15*. Vol.II. Association for Computing Machinery; 2015:147-148. doi:[10.1145/2811681.2824996](https://doi.org/10.1145/2811681.2824996)
70. Marijan D, Liaen M, Sen S. DevOps improvements for reduced cycle times with integrated test optimizations for continuous integration. In: 2018 IEEE 42nd Annual Computer Software and Applications Conference (COMPSAC). IEEE; 2018:22-27. doi:[10.1109/COMPSAC.2018.00012](https://doi.org/10.1109/COMPSAC.2018.00012)
71. Filippetto AS, Lima R, Barbosa JLV. A risk prediction model for software project management based on similarity analysis of context histories. *Inf SoftwTechnol*. 2021;131:106497. doi:[10.1016/j.infsof.2020.106497](https://doi.org/10.1016/j.infsof.2020.106497)
72. Samarawickrama SS, Perera I. Continuous scrum: a framework to enhance scrum with DevOps. In: 2017 Seventeenth International Conference on Advances in ICT for Emerging Regions (ICTer); 2017:1-7. doi:[10.1109/ICTER.2017.8257808](https://doi.org/10.1109/ICTER.2017.8257808)

73. Bruneliere H, Muttillio V, Eramo R, et al. AIDOaRT: AI-augmented automation for DevOps, a model-based framework for continuous development in cyber-physical systems. *Microprocess Microsyst.* 2022;94:104672. doi:[10.1016/j.micpro.2022.104672](https://doi.org/10.1016/j.micpro.2022.104672)
74. Angara J, Gutta S, Prasad S. DevOps with continuous testing architecture and its metrics model. In: S. and H. I. K, Kumar SMNSP, Bakshi, eds. *Recent Findings in Intelligent Computing Techniques*. Springer Singapore; 2018:271-281. doi:[10.1007/978-981-10-8633-5_28](https://doi.org/10.1007/978-981-10-8633-5_28)
75. Prado Lima JA, Vergilio SR. Test case prioritization in continuous integration environments: a systematic mapping study. *Inf SoftwTechnol.* 2020;121:106268. doi:[10.1016/j.infsof.2020.106268](https://doi.org/10.1016/j.infsof.2020.106268)
76. Çalikli G, Staron M, Meding W. Measure early and decide fast: transforming quality management and measurement to continuous deployment. In: *Proceedings of the 2018 International Conference on Software and System Process, in ICSSP'18*. Association for Computing Machinery; 2018:51-60. doi:[10.1145/3202710.3203156](https://doi.org/10.1145/3202710.3203156)
77. Chakraborty Bapi S, Karthikeyan A. Continuous monitoring and changes. In: *Understanding Azure Monitoring: Includes IaaS and PaaS Scenarios*. Apress; 2019:205-216. doi:[10.1007/978-1-4842-5130-0_6](https://doi.org/10.1007/978-1-4842-5130-0_6)
78. Rezaei J, Nispeling T, Sarkis J, Tavasszy L. A supplier selection life cycle approach integrating traditional and environmental criteria using the best worst method. *J Clean Prod.* 2016;135:577-588. doi:[10.1016/j.jclepro.2016.06.125](https://doi.org/10.1016/j.jclepro.2016.06.125)
79. Bass JM. Artefacts and agile method tailoring in large-scale offshore software development programmes. *Inf SoftwTechnol.* 2016;75:1-16. doi:[10.1016/j.infsof.2016.03.001](https://doi.org/10.1016/j.infsof.2016.03.001)
80. Li J, Moe NB, Dybå T. Transition from a plan-driven process to Scrum. In: *Proceedings of the 2010 ACM-IEEE international symposium on empirical software engineering and measurement*. ACM; Sep. 2010:1-10. doi:[10.1145/1852786.1852804](https://doi.org/10.1145/1852786.1852804)
81. Wang Z. Modelling and simulation of Scrum team strategies: a multi-agent approach. In: P, Radek PZS, Silhavy, eds. *Software Engineering Perspectives in Intelligent Systems*. Springer International Publishing; 2020:32-63. doi:[10.1007/978-3-030-63322-6_4](https://doi.org/10.1007/978-3-030-63322-6_4)
82. Punjabi R, Bajaj R. User stories to user reality: a DevOps approach for the cloud. In: *2016 IEEE International Conference on Recent Trends in Electronics, Information & Communication Technology (RTEICT)*; 2016:658-662. doi:[10.1109/RTEICT.2016.7807905](https://doi.org/10.1109/RTEICT.2016.7807905)
83. Bernhart M, Grechenig T. On the understanding of programs with continuous code reviews. In: *2013 21st International Conference on Program Comprehension (ICPC)*; 2013:192-198. doi:[10.1109/ICPC.2013.6613847](https://doi.org/10.1109/ICPC.2013.6613847)
84. Artac M, Borovssak T, Di Nitto E, Guerriero M, Tamburri DA. DevOps: introducing infrastructure-as-code. In: *2017 IEEE/ACM 39th International Conference on Software Engineering Companion (ICSE-C)*; 2017:497-498. doi:[10.1109/ICSE-C.2017.162](https://doi.org/10.1109/ICSE-C.2017.162)
85. Akhtar A, Bakhtawar B, Akhtar S. Extreme programming vs Scrum: a comparison of Agile models. *Int J Technol Innov Manag (IJTIM)*. 2022;2(2):80-96. doi:[10.54489/ijtim.v2i2.77](https://doi.org/10.54489/ijtim.v2i2.77)
86. Pendyala V. *Evolution of integration, build, test, and release engineering into DevOps and to DevSecOps*; 2020:1-20. doi:[10.4018/978-1-7998-1863-2.ch001](https://doi.org/10.4018/978-1-7998-1863-2.ch001)
87. Fernandez-Gauna B, Rojo N, Graña M. Automatic feedback and assessment of team-coding assignments in a DevOps context. *Int J Educ Technol High Educ.* 2023;20(1):17. doi:[10.1186/s41239-023-00386-6](https://doi.org/10.1186/s41239-023-00386-6)
88. Soni M. End to end automation on cloud with build pipeline: the case for DevOps in insurance industry, continuous integration, continuous testing, and continuous delivery. *IEEE Int Conf Cloud Comput Emerging Markets (CEEM)*. 2015;2015:85-89. doi:[10.1109/CEEM.2015.29](https://doi.org/10.1109/CEEM.2015.29)
89. Liu L, Xie D, Cheng Y, Li G. Architecture scheme of DevOps for cross network and multiple environment collaboration. In: *The 5th International Conference on Computer Science and Application Engineering*. ACM; 2021:1-5. doi:[10.1145/3487075.3487116](https://doi.org/10.1145/3487075.3487116)
90. Marijan D, Gotlieb A, Liaen M. A learning algorithm for optimizing continuous integration development and testing practice. *Softw Pract Exp.* 2019;49(2):192-213. doi:[10.1002/spe.2661](https://doi.org/10.1002/spe.2661)
91. Saidani I, Ouni A, Mkaouer MW, Palomba F. On the impact of continuous integration on refactoring practice: an exploratory study on TravisTorrent. *Inf SoftwTechnol.* 2021;138:106618. doi:[10.1016/j.infsof.2021.106618](https://doi.org/10.1016/j.infsof.2021.106618)
92. Sokolowski D, Weisenburger P, Salvaneschi G. Automating serverless deployments for DevOps organizations. In: *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, in ESEC/FSE 2021*. Association for Computing Machinery; 2021:57-69. doi:[10.1145/3468264.3468575](https://doi.org/10.1145/3468264.3468575)
93. Dam HK, Tran T, Grundy J, Ghose A, Kamei Y. Towards effective AI-powered Agile project management. In: *2019 IEEE/ACM 41st International Conference on Software Engineering: New Ideas and Emerging Results (ICSE-NIER)*; 2019:41-44. doi:[10.1109/ICSE-NIER.2019.00019](https://doi.org/10.1109/ICSE-NIER.2019.00019)
94. Strugar D. Complex systems: on design and architecture of adaptable dashboards. In: J.-M. and M. B, Manuel PAM, Bruel, eds. *Software Technology: Methods and Tools*. Springer International Publishing; 2019:176-186.
95. Sharma Sarika D, Kumar. On the development of feature-based sprint in AGILE. In: S. and M. K. K, Yu-Chen TMCH, Tiwari, eds. *Ambient Communications and Computer Systems*. Springer Singapore; 2019:223-235. doi:[10.1007/978-981-13-5934-7_20](https://doi.org/10.1007/978-981-13-5934-7_20)
96. de Aguiar Monteiro L, Pessoa Monteiro DSM, Carvalho Almeida WH, de Lima A, Sette IS. Methods of implementation, maturity models and definition of roles in DevOps frameworks: a systematic mapping. In: *2020 International Conference on Computational Science and Computational Intelligence (CSCI)*; 2020:1766-1773. doi:[10.1109/CSCI51800.2020.00327](https://doi.org/10.1109/CSCI51800.2020.00327)

How to cite this article: Kumar A, Nadeem M, Shameem M. Multicriteria decision-making-based framework for implementing DevOps practices: A fuzzy best-worst approach. *J Softw Evol Proc.* 2024;36(6):e2631. doi:[10.1002/smr.2631](https://doi.org/10.1002/smr.2631)

APPENDIX 1

SLR studies: <https://tinyurl.com/2nntu6jt>

APPENDIX 2

Questionnaire for empirical validation

Section 1 A (respondent information)

Full name (optional)

Job title/position

Years of experience in the field of DevOps

Organization address with country name

Email address

Team of DevOps to whom you belong

☐ Dev☐ Ops**Section 1 B (organizational details)**

Name of organization optional)

Please mention the size of your organization.

☐ Small☐ Medium☐ Large☐ Not sure

Please mention the number of employees in your organization.

☐ Less than 20☐ 21–100☐ 101–200☐ More than 200

Please mention the type of organization.

☐ National☐ Multi-national

How long has the DevOps paradigm been adopted in your organization? (in years)

Section 2—Practices for DevOps implementation

This section contains practices for implementing DevOps in software organization. Please use the given Likert scale to rate the identified practices based on your understanding and experience.

(SD, strongly disagree), (D, disagree), (N, neutral), (A, agree), (SA, strongly agree)

Practices for DevOps implementation

SD

D

N

A

SA

Architecture Specifications (P1)

Automated Unit Testing (P2)

Backlog Management (P3)

Burndown Charts (P4)

Code Review (P5)

Coding Standards (P6)

Collective Code Ownership (P7)

Automated Deployment Pipeline (P8)

Automated Integration of Code Changes (P9)

Daily Standup (P10)

Defining the Deployed Features (P11)

Estimating Team Effort (P12)

End-to-End System Testing (P13)

Iteration/Sprint Reviews (P14)

Limit work in Progress (P15)

Code Refactoring (P16)

Planning for early Release (P17)

Effective Sprint Retrospectives (P18)

User Stories-based Implementation (P19)